TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

VÕ NGUYỄN GIA HUY

MSSV: 223940

LỚP: DH22KPM02

TÊN ĐỀ TÀI
NGHIÊN CỨU VÀ XÂY DỰNG SÀN GIAO DỊCH TÀI SẢN SỐ TÍCH HỢP CÔNG NGHỆ ZERO-KNOWLEDGE PROOF

ĐỒ ÁN CHUYÊN NGÀNH

NGÀNH: KỸ THUẬT PHẦN MỀM
Mã số ngành: 7480103

Tháng 03-2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

VÕ NGUYỄN GIA HUY

MSSV: 223940

LỚP: DH22KPM02

TÊN ĐỀ TÀI
NGHIÊN CỨU VÀ XÂY DỰNG SÀN GIAO DỊCH TÀI SẢN SỐ TÍCH HỢP CÔNG NGHỆ ZERO-KNOWLEDGE PROOF

ĐỒ ÁN CHUYÊN NGÀNH

NGÀNH: KỸ THUẬT PHẦN MỀM

Mã số ngành: 7480103

CÁN BỘ HƯỚNG DẪN

ThS. ĐẶNG MẠNH HUY

Tháng 03-2026

LỜI CẢM ƠN

Trước hết, em xin gửi tới các thầy, cô Khoa Công nghệ Thông tin, lời chào trân trọng, lời chúc sức khỏe và lời cảm ơn sâu sắc. Với sự quan tâm, dạy dỗ, chỉ bảo tận tình chu đáo của thầy cô, đến nay em đã có thể hoàn thành đồ án với đề tài: "Phát triển Nghiên cứu và Xây dựng Sàn giao dịch Tài sản số tích hợp Công nghệ Zero-Knowledge Proof".

Đặc biệt, em xin gửi lời cảm ơn chân thành nhất đến thầy Đặng Mạnh Huy đã quan tâm giúp đỡ, hướng dẫn tận tình em hoàn thành tốt đồ án này trong thời gian vừa qua. Những buổi thảo luận, những lời khuyên quý báu và sự kiên nhẫn của thầy đã giúp em vượt qua nhiều khó khăn, từ việc thiết kế cơ sở dữ liệu đến tối ưu hóa giao diện ứng dụng.

Em cũng xin bày tỏ lòng biết ơn đến lãnh đạo Trường Đại học Nam Cần Thơ, các thầy cô Khoa Công nghệ Thông tin đã trực tiếp và gián tiếp giúp đỡ em trong suốt quá trình học tập và nghiên cứu đề tài. Môi trường học tập và nghiên cứu mà nhà trường và khoa tạo ra đã giúp em có điều kiện thuận lợi để hoàn thành đồ án.

Cuối cùng, em xin cảm ơn các bạn sinh viên đã chia sẻ kiến thức, kinh nghiệm và hỗ trợ em trong quá trình thực hiện. Sự hợp tác và tinh thần đoàn kết của các bạn đã giúp em vượt qua những trở ngại kỹ thuật và hoàn thiện đồ án một cách tốt nhất.

Với điều kiện thời gian và kinh nghiệm còn hạn chế, đồ án này không thể tránh khỏi những thiếu sót. Em rất mong nhận được sự chỉ bảo, đóng góp ý kiến của các thầy cô để em có điều kiện bổ sung, nâng cao kiến thức của mình, phục vụ tốt hơn công tác thực tế sau này.

Em xin chân thành cảm ơn!

Cần Thơ, ngày 17 tháng 03 năm 2026

Người thực hiện

Võ Nguyễn Gia Huy

LỜI CAM KẾT

Em xin cam kết báo cáo đồ án này được hoàn thành dựa trên các kết quả nghiên cứu, tìm hiểu và phát triển mã nguồn của chính bản thân em trong khuôn khổ của đề tài "Nghiên cứu và Xây dựng Sàn giao dịch Tài sản số tích hợp Công nghệ Zero-Knowledge Proof", dưới sự hướng dẫn của ThS. Đặng Mạnh Huy. Các trích dẫn, số liệu và tài liệu tham khảo đều được ghi chú rõ ràng, trung thực. Các kết quả kiến trúc hệ thống và thuật toán này chưa từng được sao chép hay dùng để nộp cho bất cứ đồ án cùng cấp nào trước đó. Nếu có bất kỳ sự gian lận nào, em xin hoàn toàn chịu trách nhiệm trước Hội đồng kỷ luật của Khoa và Nhà trường.

Cần Thơ, ngày 17 tháng 03 năm 2026

Người thực hiện

Võ Nguyễn Gia Huy

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày 17 tháng 03 năm 2026

Giảng Viên

ThS. Đặng Mạnh Huy

MỤC LỤC

CHƯƠNG 1 GIỚI THIỆU 1

1.1 Tổng quan đề tài 1

1.2 Mục tiêu đề tài 2

1.3 Phạm vi đề tài 3

1.4 Ý nghĩa thực tiễn 3

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT 4

2.1 Kiến trúc hệ thống giao dịch độ trễ thấp (Low-Latency Trading Systems) 4

2.1.1 Động cơ khớp lệnh (Matching Engine) và Nguyên lý Price-Time Priority 4

2.1.2 Mô hình tính toán trên bộ nhớ chính (In-Memory Processing Model) 5

2.2 Nền tảng cấu trúc dữ liệu hiệu năng cao 6

2.2.1 Cây tìm kiếm tự cân bằng (BTreeMap) trong định tuyến mức giá 6

2.2.2 Cấu trúc lưu trữ phẳng (SlotMap / Generational Arena) 7

2.2.3 Danh sách liên kết đôi nhúng (Intrusive Doubly Linked List) 7

2.2.4 Chứng minh độ phức tạp thời gian hằng số O(1) 7

2.3 Cơ sở mật mã học và Bằng chứng không tri thức (ZKP) 9

2.3.1 Khái niệm Zero-Knowledge Proof và Bằng chứng khả năng thanh toán (Proof of Solvency) 9

2.3.2 Cấu trúc Cây tổng Merkle (Merkle Sum Tree) 9

2.3.3 Hàm băm Poseidon (Poseidon Hash) 10

CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 12

3.1 Mô tả hệ thống 12

3.1.1 Giới thiệu hệ thống 12

3.1.2 Yêu cầu chức năng 12

3.1.3 Yêu cầu phi chức năng 13

3.2 Kiến trúc tổng quan 13

3.3 Quy trình hoạt động 14

3.3.1 Quy trình Đặt lệnh & Khớp lệnh 14

3.3.2 Quy trình Hủy lệnh & Hoàn tiền (Cancel Order) 15

3.3.2 Quy trình Xác thực ZKP tại Client-side 16

3.3.3 Sơ đồ Trạng thái: Vòng đời Lệnh giao dịch (Order Lifecycle) 17

3.4 Thiết kế hệ thống 18

3.4.1 Thiết kế cơ sở dữ liệu 18

3.4.2 Thiết kế sơ đồ lớp 22
3.4.3 Mô hình ERD 23
3.4.4 Thiết kế chức năng 23
3.4.5 Thiết kế kiến trúc hệ thống 26
3.4.6 Thiết kế phân quyền và bảo mật 26
CHƯƠNG 4 GIAO DIỆN HỆ THỐNG 27
4.1 Giao diện landing page 27
4.2 Giao diện Đăng Nhập 27
4.3 Giao diện Đăng ký 28
4.4 Giao diện Trade 28
4.5 Giao diện Asset 29
4.6 Giao diện Trade History 29
4.7 Giao diện ZK-Verify 30
4.8 Giao diện Dashboard Admin 30
4.9 Giao diện Market & Asset 31
4.10 Giao diện Quản Lý User 31
4.11 Giao diện ZKP Audit 32
CHƯƠNG 5 KẾT LUẬN 33
5.1 Kết luận 33
5.2 Hạn chế 33
5.3 Hướng phát triển 34
TÀI LIỆU THAM KHẢO 35

DANH SÁCH BẢNG

Bảng 3.1 Bảng users 18
Bảng 3.2 Bảng assests 18
Bảng 3.3 Bảng balances 18
Bảng 3.4 Bảng orders_log 19
Bảng 3.5 Bảng trades_log 20
Bảng 3.6 Bảng candles 20
Bảng 3.7 Danh sách API Phân hệ Quản lý Tài khoản và Ví 23
Bảng 3.8 Danh sách API Phân hệ Giao dịch lõi 24
Bảng 3.9 Danh sách API và WebSocket Phân hệ Dữ liệu thị trường 25
Bảng 3.10 Danh sách API Phân hệ Kiểm toán Minh bạch ZKP 25

DANH SÁCH HÌNH

Hình 2.1 Kiến trúc hủy lệnh 8
Hình 2.2 Kiến trúc Merkle Sum Tree 10
Hình 3.1 Sơ đồ tuần tự luồng đặt lệnh 14
Hình 3.2 Sơ đồ tuần tự luồng hủy lệnh 15
Hình 3.3 Sơ đồ tuần tự luồng ZKP 16
Hình 3.4 Sơ đồ Trạng thái của vòng đời một lệnh giao dịch 17
Hình 3.5 Sơ đồ lớp 22
Hình 3.6 Sơ đồ ERD tổng quát. 23
Hình 4.1 Giao diện Landing Page 27
Hình 4.2 Giao diện Đăng Nhập 27
Hình 4.3 Giao diện Đăng Ký 28
Hình 4.4 Giao diện Trade 28
Hình 4.5 Giao diện Asset 29
Hình 4.6 Giao diện Trade History 29
Hình 4.7 Giao diện ZK-Verify 30
Hình 4.8 Giao diện Dashboard Admin 30
Hình 4.9 Giao diện Market & Asset 31
Hình 4.10 Giao diện Quản Lý User 31
Hình 4.11 Giao diện ZKP Audit 32

DANH MỤC TỪ VIẾT TẮT

TỪ VIẾT TẮT	GIẢI NGHĨA
CEX	Centralized Exchange
DEX	Decentralized Exchange
ERD	Entity-Relationship Diagram
JWT	JSON Web Token
OHLCV	Open, High, Low, Close, Volume
PoL	Proof of Liabilities
PoR	Proof of Reserve
REST	Representational State Transfer
RPC	Remote Procedure Call
TPS	Transactions Per Second
WASM	WebAssembly
ZKP	Zero-Knowledge Proof

CHƯƠNG 1
GIỚI THIỆU

1.1 Tổng quan đề tài

Trong kỷ nguyên tài chính công nghệ và sự bùng nổ của thị trường tài sản số (*Cryptocurrency*), nhu cầu giao dịch thông qua các nền tảng điện tử đang tăng trưởng mạnh mẽ. Hiện nay, cấu trúc nền tảng giao dịch bị chia cắt bởi hai thái cực: Sàn giao dịch tập trung (*Centralized Exchange - CEX*) và Sàn giao dịch phi tập trung (*Decentralized Exchange - DEX*).

Các nền tảng DEX mang lại sự minh bạch tuyệt đối nhờ công nghệ sổ cái phân tán (*Blockchain*), nhưng lại bộc lộ những giới hạn vật lý nghiêm trọng về thông lượng xử lý (*tốc độ TPS thấp*) và rủi ro trượt giá (*Slippage*). Ngược lại, các sàn CEX tiếp tục thống trị thị trường nhờ cung cấp thanh khoản sâu và tốc độ khớp lệnh cực nhanh. Tuy nhiên, yếu điểm chí mạng của mô hình CEX là chúng vận hành như những "hộp đen" (*Blackbox*). Người dùng ủy thác tài sản cho sàn nhưng không có bất kỳ cơ sở toán học nào để xác thực liệu hệ thống có đang duy trì tỷ lệ dự trữ 1:1 đối với tài sản của họ, hay đang âm thầm chiếm dụng vốn (*Fractional Reserve*) cho các hoạt động đầu tư mạo hiểm. Sự sụp đổ chấn động của sàn giao dịch FTX là minh chứng rõ nét nhất cho rủi ro hệ thống này.

Xuất phát từ thực tiễn khắt khe đó, đề tài "Nghiên cứu và Xây dựng Sàn giao dịch Tài sản số tích hợp Công nghệ Zero-Knowledge Proof" được thực hiện nhằm giải quyết bài toán nan giải: Kết hợp tốc độ siêu tốc của CEX với sự minh bạch tuyệt đối của DEX.

Để đạt được hiệu năng ngang tầm các hệ thống High-Frequency Trading (*HFT*), hệ thống không sử dụng cơ sở dữ liệu trên ổ cứng (*Disk-based DB*) cho các thao tác nóng. Thay vào đó, đề tài xây dựng một lõi khớp lệnh (*Matching Engine*) và Sổ cái (*Ledger*) hoạt động hoàn toàn trên bộ nhớ chính (*In-Memory/RAM*). Khác với các ứng dụng thông thường chỉ dùng RAM làm bộ nhớ đệm (Cache), kiến trúc của dự án biến RAM thành "*Nguồn sự thật*" (*Primary Source of Truth*), cho phép khóa tiền và khớp lệnh với độ trễ cực thấp (*micro-giây*).

Đồng thời, để phá vỡ tính chất "*hộp đen*" của CEX, hệ thống tiên phong tích hợp công nghệ mật mã học Zero-Knowledge Proof (*ZKP*). Thông qua việc xuất bản Bằng chứng khả năng thanh toán (*Proof of Solvency*), sàn giao dịch có thể chứng minh tính toàn vẹn của tổng tài sản dự trữ bằng toán học mà hoàn toàn không làm rò rỉ thông tin định danh hay số dư cá nhân của bất kỳ người dùng nào.

1.2 Mục tiêu đề tài

Đề tài hướng tới việc nghiên cứu và hiện thực hóa các mục tiêu cốt lõi sau đây, tập trung vào việc định nghĩa lại tiêu chuẩn hiệu năng và bảo mật cho mô hình Sàn giao dịch Tập trung (*CEX*):

- Phát triển Core Matching Engine theo kiến trúc Memory-Centric (*Lấy RAM làm trung tâm*): Xây dựng một động cơ khớp lệnh hoạt động hoàn toàn trên bộ nhớ vật lý (*In-Memory*). Mục tiêu là thay đổi bản chất của RAM trong hệ thống: RAM không đóng vai trò là bộ nhớ đệm (*Cache*) như các ứng dụng web thông thường, mà chính là Nguồn sự thật duy nhất (*Primary Source of Truth*) lưu trữ toàn vẹn trạng thái Sổ cái (*Ledger*) và Sổ lệnh (*OrderBook*). Việc này giúp loại bỏ hoàn toàn độ trễ do truy xuất đĩa cứng (*Disk I/O*) khỏi luồng thực thi nóng (*Hot-path*), đạt thông lượng cao (*High throughput*) với độ trễ siêu thấp *(< 1 mili-giây*).
- Tối ưu hóa cấu trúc dữ liệu đạt độ phức tạp *O(1)* tuyệt đối: Đột phá khỏi giới hạn *O(log N)* của các cấu trúc dạng cây truyền thống khi xử lý hàng đợi lệnh. Áp dụng thiết kế hệ thống chuyên sâu kết hợp giữa *SlotMap* (*Generational Arena*) và *Intrusive Doubly Linked List* (*Danh sách liên đôi kép nhúng*) để đảm bảo mọi thao tác quan trọng nhất - bao gồm thêm lệnh, khớp lệnh và hủy lệnh ở vị trí bất kỳ - đều được thực thi với độ phức tạp thời gian hằng số *O(1)* và độ trễ có thể dự đoán được (*Deterministic Latency*).
- Nghiên cứu và ứng dụng mật mã học Zero-Knowledge Proof (*ZKP*): Ứng dụng công nghệ Bằng chứng không tri thức để giải quyết bài toán niềm tin của CEX. Xây dựng hệ thống Bằng chứng nợ phải trả (*Proof of Liabilities*) thông qua cấu trúc *Merkle Sum Tree* và hàm băm *Poseidon* (*ZK-Friendly*), cho phép sàn minh bạch năng lực thanh toán mà vẫn bảo vệ tính riêng tư, không tiết lộ thông tin định danh hay số dư cá nhân của khách hàng.
- Phát triển cơ chế xác minh phi tín nhiệm (*Trustless Verification*) tại Client-side: Xây dựng luồng kiểm toán độc lập cho phép người dùng tải Bằng chứng cá nhân (*Merkle Path*) và chạy mã *WebAssembly* (*WASM*) trực tiếp trên trình duyệt. Mục tiêu là trao quyền tự kiểm chứng toán học cho người dùng mà không cần đặt niềm tin (*Trustless*) vào máy chủ trung tâm.
- Thiết kế luồng dữ liệu bất đồng bộ (*Async Persistence*): Đảm bảo tính bền vững của dữ liệu tài chính (*Data Durability*) mà không hy sinh tốc độ khớp lệnh. Sử dụng cơ chế hàng đợi (*MPSC Channel*) và tiến trình

chạy nền (*Background Worker*) để ghi nhận nhật ký giao dịch xuống cơ sở dữ liệu vật lý (*PostgreSQL*) một cách bất đồng bộ.

1.3 Phạm vi đề tài

Do giới hạn về thời gian nghiên cứu và tài nguyên phần cứng, đồ án tập trung định hình và giải quyết chuyên sâu các bài toán kiến trúc lõi ở tầng Backend, cấu trúc dữ liệu trên RAM và luồng dữ liệu mật mã học. Cụ thể như sau:

- Bao gồm: Tập trung xây dựng lõi khớp lệnh và Sổ cái *In-Memory* (đặt, khớp, hủy lệnh); phát triển thuật toán sinh Bằng chứng khả năng thanh toán (*Proof of Solvency*) dựa trên *Merkle Sum Tree* và hàm băm *Poseidon*; cùng giao diện *SPA* cho phép người dùng giao dịch và tự kiểm chứng bằng *WebAssembly*.
- Không bao gồm: Lược bỏ việc tích hợp trực tiếp với các mạng Blockchain (chỉ dùng nạp/rút giả lập), không xử lý định danh *KYC/AML*, không phát triển giao dịch phái sinh (*Futures/Margin*) và kiến trúc Layer 2 để tập trung toàn bộ tài nguyên tối ưu hiệu năng cho Core Engine.

1.4 Ý nghĩa thực tiễn

Sản phẩm của đồ án mang lại một mô hình kiến trúc tham khảo chuẩn mực cho các hệ thống tài chính thế hệ mới (*Fintech 2.0*).

Đề tài giải quyết trực diện một bài toán nhức nhối trong ngành công nghiệp Blockchain và chứng minh một luận điểm quan trọng trong thiết kế hệ thống: Tính minh bạch và bảo mật bằng toán học không nhất thiết phải đánh đổi bằng sự suy giảm hiệu năng khớp lệnh.

Bằng cách tiên phong ứng dụng ZKP vào cấu trúc Sổ cái *In-Memory* để tạo ra mô hình "*Sàn giao dịch tập trung có thể kiểm chứng*" (*Verifiable CEX*), đồ án cung cấp một giải pháp công nghệ kỹ thuật cao giúp các tổ chức tài chính tập trung (*CEX*) khôi phục và củng cố vững chắc niềm tin của khách hàng, đặc biệt là sau những cuộc khủng hoảng chiếm dụng vốn làm rung chuyển thị trường.

CHƯƠNG 2
CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc hệ thống giao dịch độ trễ thấp (Low-Latency Trading Systems)

2.1.1 Động cơ khớp lệnh (Matching Engine) và Nguyên lý Price-Time Priority

Động cơ khớp lệnh (*Matching Engine*) được ví như "trái tim" của mọi nền tảng giao dịch tài chính điện tử, từ thị trường chứng khoán truyền thống đến các sàn giao dịch tài sản số. Về mặt bản chất kỹ thuật, nó là một cỗ máy trạng thái (*State Machine*) hoạt động liên tục nhằm tiếp nhận luồng lệnh mua (*Bids*) và lệnh bán (*Asks*) từ người dùng, sau đó tiến hành bắt cặp và thực thi giao dịch dựa trên các quy tắc toán học mang tính xác định (*Deterministic*). Đối với một hệ thống lõi đạt chuẩn, Matching Engine phải đáp ứng đồng thời hai điều kiện tiên quyết: tuyệt đối không để xảy ra tình trạng chi tiêu kép (*Double-spending*) gây thiếu hụt tài sản, và phải duy trì được thông lượng xử lý (*Throughput*) đạt mức hàng chục ngàn lệnh mỗi giây (*TPS*).

Để giải quyết bài toán phân bổ thanh khoản, hệ thống áp dụng nguyên lý Ưu tiên Giá - Thời gian (*Price-Time Priority*). Đây là bộ quy luật nền tảng nhằm đảm bảo tính công bằng tối thượng và tính minh bạch trong quá trình thực thi lệnh của tất cả các thành viên tham gia thị trường:

- *Ưu tiên về giá (Price Priority):* Thuật toán luôn ưu tiên thực thi các lệnh có mức giá có lợi nhất cho thị trường. Ở phía phe mua (Bids), khách hàng đặt mức giá mua cao nhất sẽ được hệ thống xếp lên đầu hàng đợi. Ngược lại, ở phía phe bán (Asks), người dùng chào bán với mức giá thấp nhất (rẻ nhất) sẽ được hệ thống ưu tiên khớp trước.
- *Ưu tiên về thời gian (Time Priority - FIFO):* Trong các trường hợp thanh khoản dày đặc, nhiều cá nhân có thể cùng đặt lệnh tại một mức giá giống hệt nhau (Price Level). Khi đó, nguyên tắc "*Vào trước, Ra trước*" (*First In, First Out*) sẽ được kích hoạt nhằm giải quyết xung đột. Lệnh giao dịch nào được truyền tải đến máy chủ sớm hơn - xác định thông qua dấu thời gian (*Timestamp*) hoặc số thứ tự định tuyến tuần tự (*Sequence ID*) - sẽ được ưu tiên phục vụ trước.

Sự kết hợp chặt chẽ của hai nguyên lý này tạo ra một hàng đợi hai chiều có tính thứ tự nghiêm ngặt. Khi một lệnh thị trường (*Taker Order*) chủ động tiến vào hệ thống, thuật toán khớp lệnh sẽ quét qua cấu trúc sổ lệnh (*Walking the book*) và "ăn" dần các lệnh chờ (*Maker Orders*) tuân thủ đúng trật tự Giá - Thời gian cho đến khi khối lượng yêu cầu được lấp đầy hoàn toàn.

2.1.2 Mô hình tính toán trên bộ nhớ chính (In-Memory Processing Model)

Khi đánh giá cấu trúc hạ tầng của một phần mềm, một quan niệm sai lầm phổ biến là "mọi ứng dụng đều chạy trên RAM". Sự khác biệt đẳng cấp giữa một ứng dụng phân tán thông thường (*Traditional Web Application*) và một hệ thống Giao dịch Tần suất cao (*High-Frequency Trading - HFT*) không nằm ở việc bộ nhớ *RAM* có được sử dụng hay không, mà nằm ở định nghĩa về *Nguồn sự thật (Primary Source of Truth)* và khả năng kiểm soát *Độ trễ xác định (Deterministic Latency)*.

- *Hạn chế của Kiến trúc tập trung vào Đĩa cứng (Disk-Centric Architecture):* Trong các ứng dụng tiêu chuẩn, cơ sở dữ liệu vật lý (như PostgreSQL, MySQL) được coi là "*Nguồn sự thật*". Bộ nhớ RAM chỉ đóng vai trò là nơi thực thi mã lệnh hoặc bộ nhớ đệm (*Cache*) tạm thời. Khi một giao dịch tài chính xảy ra, CPU bắt buộc phải thực hiện các lệnh gọi I/O xuống đĩa cứng để ghi nhận thay đổi. Quá trình này tạo ra nút thắt cổ chai (*Bottleneck*) nghiêm trọng do phải chờ đợi độ trễ mạng, khóa bảng (*Table Locks*) và cơ chế đồng bộ đĩa (*Disk Sync*). Hậu quả là thời gian xử lý dao động bất định từ vài mili-giây đến hàng trăm mili-giây (*Unpredictable Latency*), hoàn toàn không thể đáp ứng được khối lượng hàng chục nghìn lệnh đổ vào sàn trong chớp mắt.
- Kiến trúc lấy Bộ nhớ làm trung tâm (*Memory-Centric Ledger*): Để phá vỡ rào cản trên, lõi Matching Engine của hệ thống được thiết kế theo kiến trúc *In-Memory*. Toàn bộ cấu trúc Sổ cái chứa tiền (*Ví*) và Sổ lệnh (*Orderbook*) được đưa thẳng lên và duy trì trực tiếp trên bộ nhớ RAM. Lúc này, RAM chính thức trở thành "*Nguồn sự thật*" của hệ thống. Mọi thao tác trừ tiền, khóa số dư và khớp lệnh đều được thực hiện thông qua khóa bộ nhớ (*Mutex/RwLock*) ở mức độ vi xử lý, hoàn toàn không cần chờ đợi truy vấn Database.
- Kiểm soát Độ trễ bằng Ngôn ngữ cấp Hệ thống (System-Level Language): Một ứng dụng *In-Memory* được viết bằng Java hay C# vẫn có thể gặp phải độ trễ do cơ chế tự động giải phóng bộ nhớ (*Garbage Collector - GC*). Khi Sổ lệnh phình to với hàng triệu lệnh, tiến trình GC sẽ quét qua RAM để dọn rác, gây ra hiện tượng "*Stop-the-world*" (đóng băng toàn bộ hệ thống trong vài chục mili-giây). Để loại bỏ triệt để rủi ro này, Core Matching Engine được hiện thực hóa bằng ngôn ngữ cấp hệ thống Rust. Sự vắng mặt của GC giúp hệ thống loại bỏ hoàn toàn hiện tượng "*Stop-the-world*", qua đó duy trì tốc độ khớp lệnh ổn định ở mức micro-giây.

- Đảm bảo tính bền vững với Xử lý I/O Bất đồng bộ (*Async Persistence*): Để đảm bảo dữ liệu giao dịch không bị mất mát khi máy chủ gặp sự cố mất điện (*Data Durability*) mà vẫn không làm chậm luồng khớp lệnh nóng (*Hot-path*), hệ thống áp dụng kỹ thuật phân tách luồng (*Thread Decoupling*):
    + Luồng Engine chỉ hoạt động thuần túy trên RAM. Khi một lệnh được khớp thành công, nó lập tức đóng gói dữ liệu thành một Sự kiện (*Event*) và ném vào kênh truyền MPSC (*Multi-Producer, Single-Consumer*). Thao tác này là *Non-blocking* (Không chờ đợi).
    + Ở phía bên kia của hệ thống, một tiến trình nền (*Background Worker*) liên tục lắng nghe từ đầu ra của kênh MPSC, gom các sự kiện thành từng cụm (*Batch Processing*) và thực hiện lệnh INSERT xuống cơ sở dữ liệu PostgreSQL.

Cơ chế này giúp Core Engine hoàn toàn được giải phóng, đạt thông lượng tối đa mà vẫn lưu vết (*Audit*) giao dịch một cách an toàn.

2.2 Nền tảng cấu trúc dữ liệu hiệu năng cao

Để duy trì tốc độ xử lý hàng chục nghìn lệnh mỗi giây (*High-Frequency Trading*), một hệ thống lưu trữ trên *RAM* thuần túy là chưa đủ nếu thuật toán bên dưới đi kèm với các vòng lặp tìm kiếm tuyến tính *O(N).* Thách thức kỹ thuật lớn nhất của một Sổ lệnh (*OrderBook*) nằm ở thao tác Hủy lệnh ở giữa hàng đợi (*Arbitrary Cancel*). Nếu sử dụng mảng tĩnh (*Array*) hoặc hàng đợi hai đầu (*VecDeque*) thông thường, việc rút một lệnh ra khỏi hàng sẽ buộc *CPU* phải quét qua toàn bộ danh sách để tìm kiếm, sau đó dịch chuyển hàng loạt các vùng nhớ phía sau lên để lấp chỗ trống. Quá trình này tạo ra độ trễ cực kỳ lớn (*Latency Spikes*) khi thị trường biến động mạnh.

Để giải quyết triệt để nút thắt cổ chai này, hệ thống áp dụng một kiến trúc dữ liệu lai (*Hybrid Data Structure*) vô cùng tinh vi, bao gồm:

2.2.1 Cây tìm kiếm tự cân bằng (BTreeMap) trong định tuyến mức giá

Hệ thống lưu trữ các mức giá (*Price Levels*) trong một cấu trúc cây tìm kiếm nhị phân tự cân bằng (*BTreeMap*).

- Đối với danh sách Chào bán (*Asks*), cây sắp xếp giá trị theo thứ tự tăng dần. Mức giá bán rẻ nhất (*Best Ask*) luôn nằm ở cực trái của gốc cây.
- Đối với danh sách Chào mua (*Bids*), cây sắp xếp theo thứ tự giảm dần. Mức giá mua cao nhất (*Best Bid*) nằm ở cực trái.

Việc sử dụng *BTreeMap* thay vì *HashMap* mang lại lợi thế sống còn: Hệ thống có thể lấy ra "*Mức giá tốt nhất*" (*Top of Book*) để khớp lệnh mà không cần qua bất kỳ bước duyệt vòng lặp nào (đạt *O(1)).* Thao tác tạo mới một mức giá chỉ mất *O(log P)* (với P là số lượng mức giá đang tồn tại), cực kỳ thân thiện với bộ nhớ đệm (*Cache-friendly*).

2.2.2 Cấu trúc lưu trữ phẳng (SlotMap / Generational Arena)

Thay vì cấp phát bộ nhớ động rải rác trên RAM cho từng Lệnh (*Order*) bằng con trỏ truyền thống, toàn bộ lệnh của hệ thống được quản lý tập trung bên trong một Cấu trúc *Generational Arena* mang tên SlotMap.

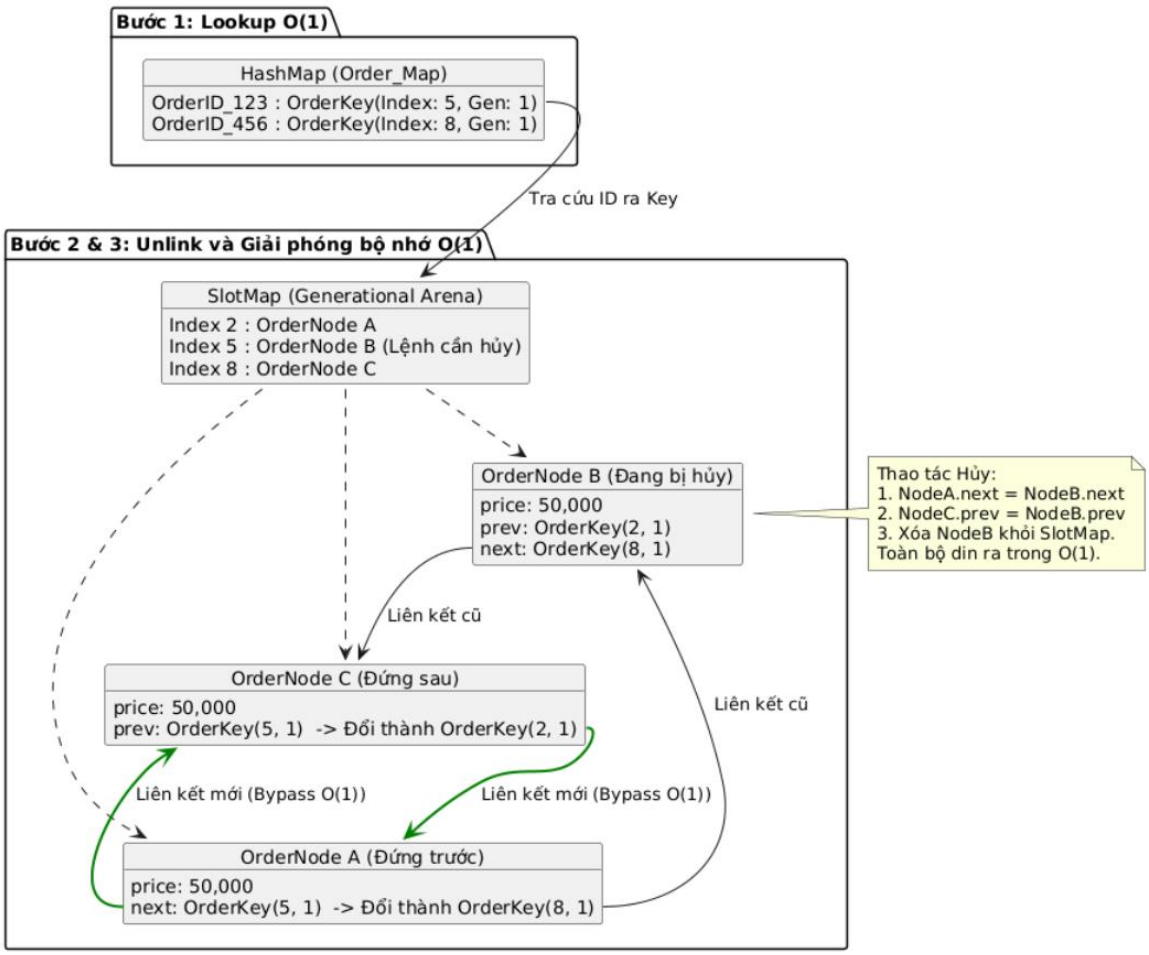
- *SlotMap* cấp phát một mảng vật lý liên tục, giúp dữ liệu các lệnh nằm sát nhau trên RAM, tối ưu hóa tối đa hiện tượng trượt bộ nhớ đệm (Cache Miss).
- Để chống lại lỗi cấp phát sai vùng nhớ (*Lỗi ABA - ABA Problem*) vốn là "*ác mộng*" của các hệ thống *HFT, SlotMap* đính kèm một bộ đếm thế hệ (*Generation Counter*) vào mỗi *Key*. Khi một lệnh bị hủy và ô nhớ đó được tái sử dụng cho một lệnh mới, hệ thống sẽ tăng "*số đời*" lên. Nhờ đó, thao tác chèn (Insert) và xóa (Remove) dữ liệu lõi luôn được thực thi an toàn tuyệt đối với chi phí thời gian hằng số *O(1).*

2.2.3 Danh sách liên kết đôi nhúng (Intrusive Doubly Linked List)

Để giữ vững nguyên tắc Ưu tiên Thời gian (*Time Priority - FIFO*) tại cùng một mức giá, hệ thống không sử dụng con trỏ (*Pointer*) chứa địa chỉ RAM. Thay vào đó, nó triển khai Danh sách liên kết kép nhúng (*Intrusive DLL*). Mỗi cấu trúc dữ liệu OrderNode được "nhúng" thêm hai thuộc tính *prev_key* và *next_key*. Các khóa này trỏ đến số thứ tự vật lý (*Index*) của lệnh đứng trước và đứng sau nó bên trong SlotMap. Thiết kế này giúp việc quản lý hàng đợi trở nên liền mạch, không sinh ra chi phí phân mảnh bộ nhớ (*Memory Fragmentation*).

2.2.4 Chứng minh độ phức tạp thời gian hằng số O(1)

Sự kết hợp của bộ ba cấu trúc dữ liệu trên tạo ra một cỗ máy khớp lệnh có khả năng thực thi các thao tác "nóng" (*Hot-path*) với độ trễ siêu thấp và hoàn toàn độc lập với số lượng lệnh đang có trên sàn. Luận điểm chứng minh toán học cho thao tác Hủy lệnh (*Cancel Order*) diễn ra như sau:



Hình 2.1 Kiến trúc hủy lệnh

1. Định tuyến Lệnh (*Lookup*): Người dùng gửi yêu cầu hủy bằng *OrderID*. Hệ thống tra cứu *ID* này trong một order_map (*HashMap*) nội bộ. Phép toán băm trả về OrderKey chứa vị trí vật lý của lệnh trong *SlotMap*. Quá trình này mất *O(1).*

2. Cắt đứt liên kết (*Unlink Node*): Dựa vào *OrderKey*, Engine truy cập thẳng vào vùng nhớ của lệnh trong *SlotMap* (*O(1))* để lấy ra *prev_key* và *next_key*. Hệ thống tiến hành cập nhật con trỏ next của lệnh đứng trước và con trỏ prev của lệnh đứng sau để nối chúng lại với nhau (*Bypass*). Quá trình viết lại 2 tham chiếu này mất chính xác 2 phép tính, tương đương *O(1).*

3. Giải phóng bộ nhớ (*Free Slot*): Lệnh hiện tại bị gỡ bỏ khỏi *SlotMap* (*O(1)),* ô nhớ được đưa vào danh sách rảnh rỗi (*Free List*) để tái sử dụng ngay lập tức.

4. Cập nhật Mức giá (*Level Pruning*): Nếu thao tác xóa làm danh sách lệnh tại mức giá đó trống rỗng, Mức giá (*Price Node*) đó sẽ bị gỡ khỏi *BTreeMap* với chi phí *O(log P)* ở *Background*.

=> Kết luận: Tổng thời gian để tìm, gỡ bỏ liên kết và tiêu hủy một lệnh giao dịch bất kỳ - ngay cả khi nó nằm ở giữa một hàng đợi chứa hàng triệu lệnh - luôn là một hằng số giới hạn *O(1).*

2.3 Cơ sở mật mã học và Bằng chứng không tri thức (ZKP)

Hệ thống giao dịch hiệu năng cao có thể giải quyết bài toán về thông lượng và độ trễ, nhưng không thể tự mình giải quyết bài toán về niềm tin (*Trust*). Để phá bỏ tính chất "*hộp đen*" của mô hình Sàn giao dịch tập trung (*CEX*), hệ thống tích hợp công nghệ mật mã học tiên tiến nhằm cung cấp các bằng chứng toán học công khai và có thể tự kiểm chứng (*Verifiable*).

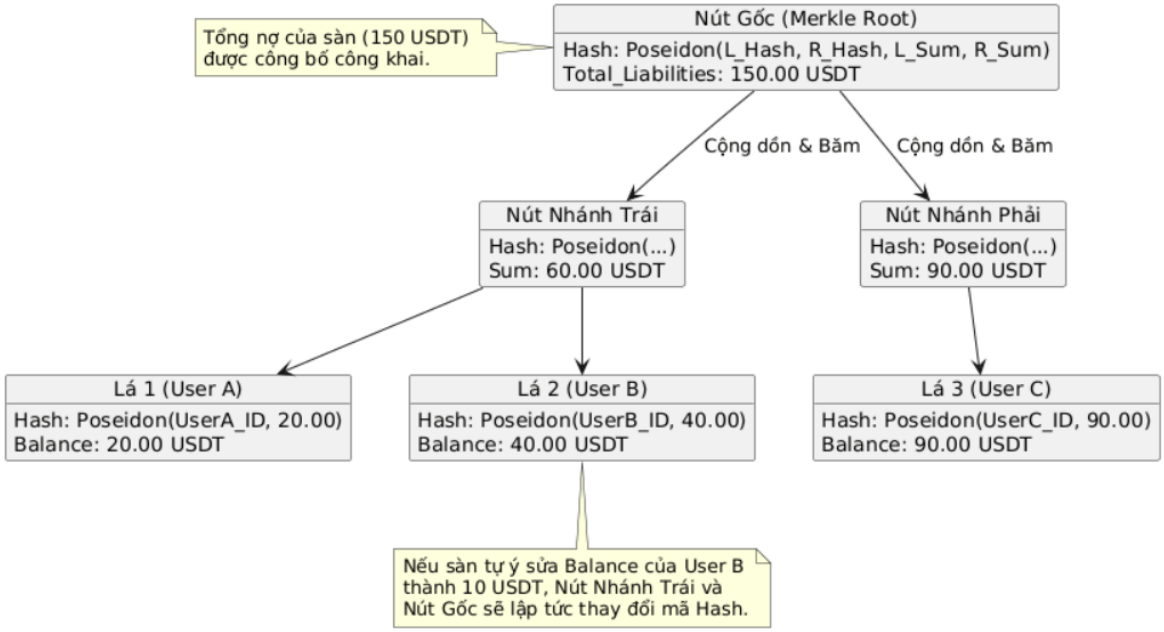
2.3.1 Khái niệm Zero-Knowledge Proof và Bằng chứng khả năng thanh toán (Proof of Solvency)

Zero-Knowledge Proof (*ZKP - Bằng chứng không tri thức*) là một giao thức mật mã học, trong đó một bên (*Prover - Người chứng minh*) có thể thuyết phục một bên khác (*Verifier - Người xác minh*) rằng một mệnh đề là ĐÚNG, mà hoàn toàn không tiết lộ bất kỳ thông tin chi tiết nào ngoài sự thật rằng mệnh đề đó là đúng.

Áp dụng vào lĩnh vực tài chính, Sàn giao dịch (*Prover*) có thể chứng minh cho Khách hàng (*Verifier*) rằng "*Tổng tài sản của tôi lớn hơn tổng số tiền tôi nợ mọi người*", nhưng tuyệt đối không cho biết danh sách tên khách hàng hay số dư trong ví của họ. Khái niệm này được gọi là Bằng chứng khả năng thanh toán (*Proof of Solvency - PoS*), được định nghĩa bằng công thức: *Proof of Solvency = Proof of Reserve (Bằng chứng dự trữ) + Proof of Liabilities* (*Bằng chứng nợ phải trả*). Trọng tâm của kiến trúc *ZKP* trong đồ án này là giải quyết vế thứ hai: Xây dựng hệ thống Chứng minh nợ (*Proof of Liabilities*) minh bạch. Nhiệm vụ của hệ thống là chứng minh: *Tổng tiền trong Ví Lạnh của sàn ≥ Tổng số dư của toàn bộ User.*

2.3.2 Cấu trúc Cây tổng Merkle (Merkle Sum Tree)

Để thực hiện *Proof of Solvency* mà không làm lộ dữ liệu quyền riêng tư, hệ thống không sử dụng Cây *Merkle* thông thường (chỉ băm dữ liệu) mà triển khai một biến thể đặc thù là *Merkle Sum Tree*, được xây dựng theo hướng từ dưới lên (Bottom-up).



Hình 2.2 Kiến trúc Merkle Sum Tree

- Nút lá (*Leaf Nodes*): Mỗi lá đại diện cho trạng thái tài sản của một người dùng duy nhất. Nó chứa hai thành phần: Giá trị băm định danh *Hash(User_ID, Balance)* và con số thể hiện *Balance* thực tế của người đó.
- Nút nhánh (*Internal Nodes*): Giá trị Hash của nút nhánh được kết hợp từ 2 con của nó: $Hash_{parent} = Poseidon(Hash_{left}, Hash_{right}, Sum_{left}, Sum_{right})$. Kèm theo đó là biến tích lũy: $Sum_{parent} = Sum_{left} + Sum_{right}$.
- Nút gốc (*Merkle Root*): Nút trên cùng chứa 1 mã *Hash* đại diện cho tính toàn vẹn của cả cây, và 1 con số chính là Tổng nợ (*Total Liabilities*) của toàn bộ sàn giao dịch.

Thông qua cấu trúc này, bất kỳ hành vi cắt xén hay gian lận số dư nào ở tầng lá đều sẽ làm thay đổi hoàn toàn mã băm tại Nút gốc. Người dùng chỉ cần được cấp đường dẫn Merkle Path cá nhân là có thể tự xác minh số dư của mình đã được cộng dồn vào Tổng nợ một cách hợp lệ.

2.3.3 Hàm băm Poseidon (Poseidon Hash)

Việc lựa chọn thuật toán băm đóng vai trò quyết định đến tính khả thi của giao thức *ZKP*. Thay vì dùng *SHA-256* (rất tốn kém chi phí tính toán khi chạy trong các mạch *ZK Circuit*), hệ thống sử dụng *Poseidon Hash*.

Khác với các hàm băm thao tác trên bit truyền thống, *Poseidon* là thuật toán băm *ZK-Friendly* tối ưu nhất hiện nay, dựa trên các phép tính cộng ma trận trên trường hữu hạn (*Finite Field*). Đặc tính này giúp giảm thiểu tối đa số lượng ràng buộc (*Constraints*) khi biên dịch cấu trúc dữ liệu thành các Mạch chứng minh (*ZK*

Circuits). Nhờ sự tối ưu của *Poseidon*, công tác sinh và xác minh bằng chứng *ZKP* diễn ra trong trình duyệt web một cách mượt mà và cực kỳ nhanh chóng. Khi người dùng nhấn nút "*Verify*", hệ thống *Client-side* sẽ tải một module *WebAssembly* (*WASM*) để tự động tính toán lại hàm băm *Poseidon* theo *Merkle Path* và đối chiếu với *Root Hash,* tạo ra một luồng kiểm toán hoàn toàn phi tín nhiệm (*Trustless*).

CHƯƠNG 3
PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Mô tả hệ thống

3.1.1 Giới thiệu hệ thống

Hệ thống được thiết kế và xây dựng để trở thành một sàn giao dịch tài sản số tập trung (*Centralized Exchange - CEX*), cung cấp khả năng tiếp nhận và khớp lệnh với độ trễ cực thấp (*micro-second latency*). Điểm đột phá của hệ thống so với các nền tảng *CEX* truyền thống là việc tích hợp sâu công nghệ mật mã học *Zero-Knowledge Proof (ZKP).* Cấu trúc này cho phép sàn giao dịch định kỳ xuất bản Bằng chứng khả năng thanh toán (*Proof of Solvency*), qua đó minh bạch hóa dự trữ tài chính cho cộng đồng người dùng mà hoàn toàn không xâm phạm hay làm lộ dữ liệu số dư thực sự của bất kỳ cá nhân nào.

Mục tiêu cốt lõi của đề tài là kiến tạo một nền tảng giao dịch không chỉ đáp ứng tiêu chuẩn tốc độ cực kỳ nhanh chóng của môi trường *High-Frequency Trading,* mà còn đảm bảo sự an toàn tuyệt đối về mặt bằng chứng dòng tiền, ngăn chặn triệt để rủi ro tổ chức quản lý lạm dụng tài sản của người dùng. Để đạt được điều này, nền tảng đã được tối ưu hóa triệt để từ tầng kiến trúc ứng dụng cho đến các cấu trúc dữ liệu lõi trên bộ nhớ RAM nhằm đáp ứng lưu lượng xử lý dữ liệu phức tạp.

3.1.2 Yêu cầu chức năng

Hệ thống được phân rã thành 3 nhóm nghiệp vụ chức năng lõi nhằm đáp ứng đầy đủ vòng đời của một giao dịch tài sản số:

Nhóm Giao dịch: Cung cấp các *Endpoint (API)* cho phép người dùng đặt lệnh giới hạn *(Limit Buy/Sell)* và thực hiện hủy lệnh (*Cancel*). Đồng thời, hệ thống cung cấp các luồng dữ liệu thời gian thực (*Real-time data stream*) để hiển thị chiều sâu Sổ lệnh (*Orderbook Depth*), dữ liệu biểu đồ nến (*OHLCV*) và lịch sử khớp lệnh công khai (*Recent Trades*).

Nhóm Quản lý tài sản: Cho phép truy xuất trạng thái ví người dùng với cơ chế phân tách nghiêm ngặt giữa Số dư khả dụng (*Free Balance*) và Số dư đang bị khóa cho các lệnh chờ (*Locked Balance*). Phân hệ này cũng hỗ trợ tính năng nạp tiền giả lập (*Mock Deposit*) nhằm mục đích cấp vốn thử nghiệm trên môi trường phát triển.

Nhóm Kiểm toán *ZKP*: Cung cấp giao diện công khai để xem xét *Proof of Solvency* của toàn bộ sàn giao dịch. Hỗ trợ người dùng tải về tệp tin *JSON* chứa Bằng chứng cá nhân (*User Inclusion Proof*) và cung cấp môi trường thực thi mã *WebAssembly* (*WASM*) để họ tự kiểm chứng tính toàn vẹn của dữ liệu trực tiếp trên trình duyệt (*Client-side*).

3.1.3 Yêu cầu phi chức năng

Độ trễ hệ thống (Ultra-low Latency): Các tác vụ quan trọng nằm trong luồng nóng (*Hot-path*) bao gồm thao tác khóa quỹ (*Reserve*) và thuật toán khớp lệnh (*Matching*) bắt buộc phải được thực thi hoàn toàn trên bộ nhớ đệm (*In-Memory*) với độ trễ xác định dưới *1 mili-giây (< 1ms*).

Tính toàn vẹn dữ liệu và Định lượng tài chính (ACID): Hệ thống được thiết kế với các khóa bảo vệ vùng nhớ nghiêm ngặt, tuyệt đối không cho phép xảy ra hiện tượng chi tiêu kép (*Double-spending*) hoặc hành vi tự khớp lệnh của chính mình (*Wash-trading*).

3.2 Kiến trúc tổng quan

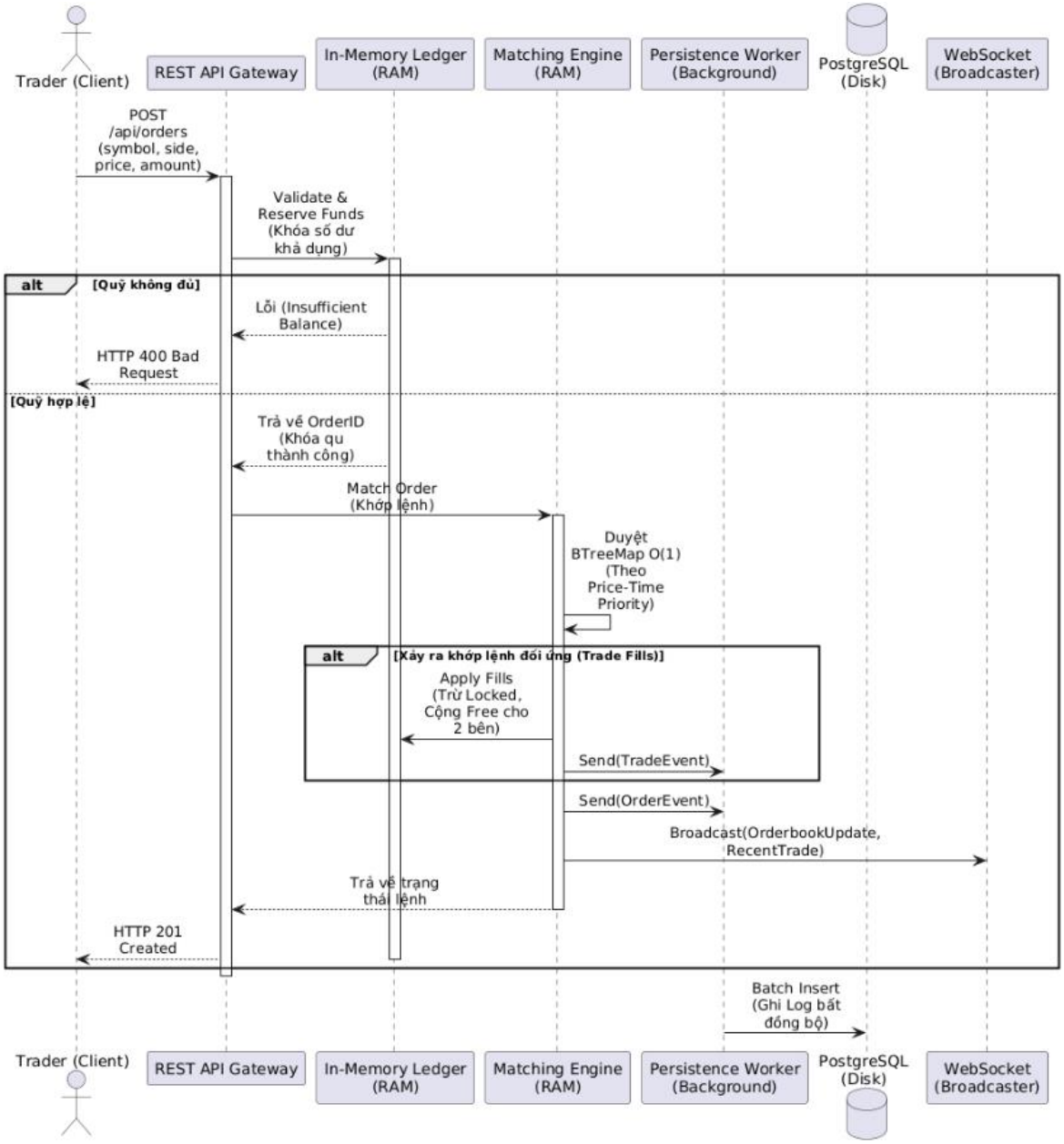
Kiến trúc hệ thống được thiết kế theo mô hình *Nguyên khối tập trung (Monolithic Core)* kết hợp với cơ chế *Xử lý I/O Bất đồng bộ (Async I/O)*. Lựa chọn kiến trúc này nhằm triệt tiêu độ trễ mạng (*Network Latency*) giữa các vi dịch vụ (*Microservices*), tối đa hóa khả năng tận dụng bộ nhớ cache của *CPU* (*Cache Locality*) cho động cơ khớp lệnh. Hệ thống được cấu thành từ ba lớp kiến trúc chính:

- Lõi Xử lý *In-Memory* (*In-Memory Ledger & Engine*): Đây là trái tim của toàn bộ nền tảng. Toàn bộ Sổ cái tài khoản (*Ledger*) và Sổ lệnh (*OrderBook*) được tải và quản lý trực tiếp trên vùng nhớ *RAM* liên tục. Lớp này tiếp nhận các yêu cầu khóa quỹ (*Reserve*) và thực thi thuật toán khớp lệnh *Price-Time Priority*. Bằng việc sử dụng các cấu trúc dữ liệu tối ưu (*SlotMap, BTreeMap*) và bảo vệ bằng khóa độc quyền (*RwLock/Mutex*), luồng thực thi nóng (*Hot-path*) hoàn toàn không bị chặn (*Zero-blocking*) bởi bất kỳ lời gọi ngoại vi nào tới cơ sở dữ liệu.
- *Pub/Sub & WebSocket*: Thay vì bắt người dùng (*Client*) phải liên tục gửi yêu cầu truy vấn (*Polling*), hệ thống sử dụng giao thức *WebSocket* hoạt động trên cơ chế *Pub/Sub*. Ngay khi lõi *In-Memory* thực thi xong một thay đổi trạng thái (*ví dụ: khớp một lệnh hoặc cập nhật chiều sâu OrderBook*), một thông điệp lập tức được đẩy qua kênh truyền (*Channel*) ra Gateway và phát sóng (*Broadcast*) đến tất cả các phiên kết nối đang mở. Điều này đảm bảo giao diện đồ họa (*UI*) của người dùng được cập nhật trạng thái đồng bộ với độ trễ tính bằng *mili-giây*.
- *Async Persistence Layer*: Để duy trì tính toàn vẹn và lưu trữ lịch sử kiểm toán mà không làm chậm Core Engine, hệ thống triển khai một Tiến trình chạy nền (*Background Worker*). Tiến trình này liên tục tiêu thụ (*Consume*) các sự kiện được đẩy ra từ lõi *RAM*, thực hiện gom nhóm thành các lô (*Batching*) và ghi chép bất đồng bộ (*Flush*) xuống hệ quản

trị cơ sở dữ liệu PostgreSQL. Cơ sở dữ liệu vật lý lúc này không còn là nơi quyết định luồng giao dịch, mà lùi về đóng vai trò là "*Sổ lưu trữ lịch sử*" phục vụ truy xuất và trích xuất Bằng chứng khả năng thanh toán (*ZKP Snapshot*).

3.3 Quy trình hoạt động

3.3.1 Quy trình Đặt lệnh & Khớp lệnh

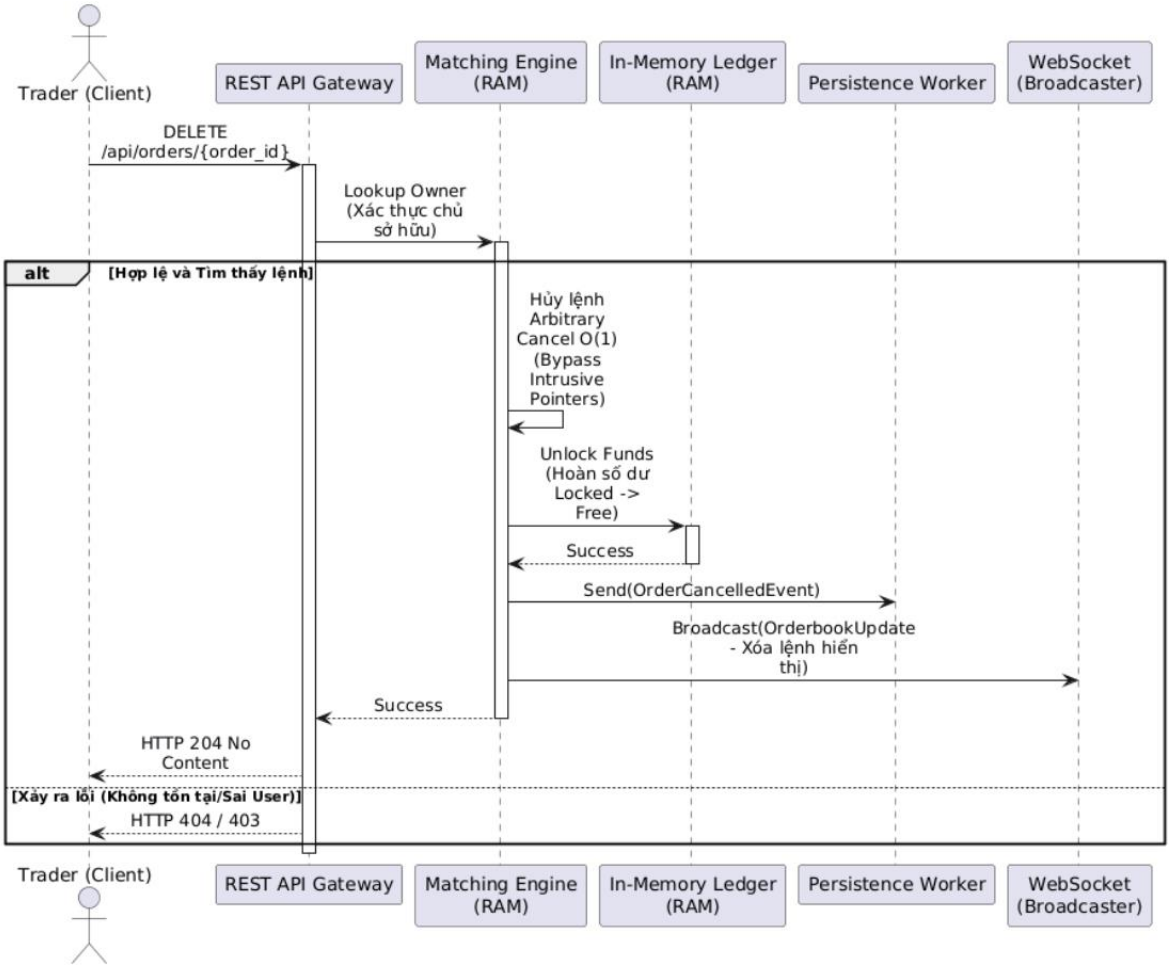


Hình 3.1 Sơ đồ tuần tự luồng đặt lệnh

1. Người dùng gửi lệnh (*REST API*).
2. *Gateway* kiểm tra định danh. Nếu hợp lệ, chuyển xuống *In-Memory Ledger*.

3. Ledger kiểm tra số dư. Nếu đủ, thực hiện Trừ *Free Balance*, Cộng vào *Locked Balance*.
4. Lệnh bay vào *Engine*. Nếu giá chạm nhau thì sinh ra *Trade* (*Trừ Locked của bên Mua, cộng Free cho bên Bán*).
5. Nếu chưa khớp đủ thì Lệnh nằm lại trên cấu trúc *BTreeMap* (*Orderbook*).
6. *Gateway* trả về *HTTP 200* OK. Hệ thống ngầm bắn *Ticker* qua *WebSocket*.

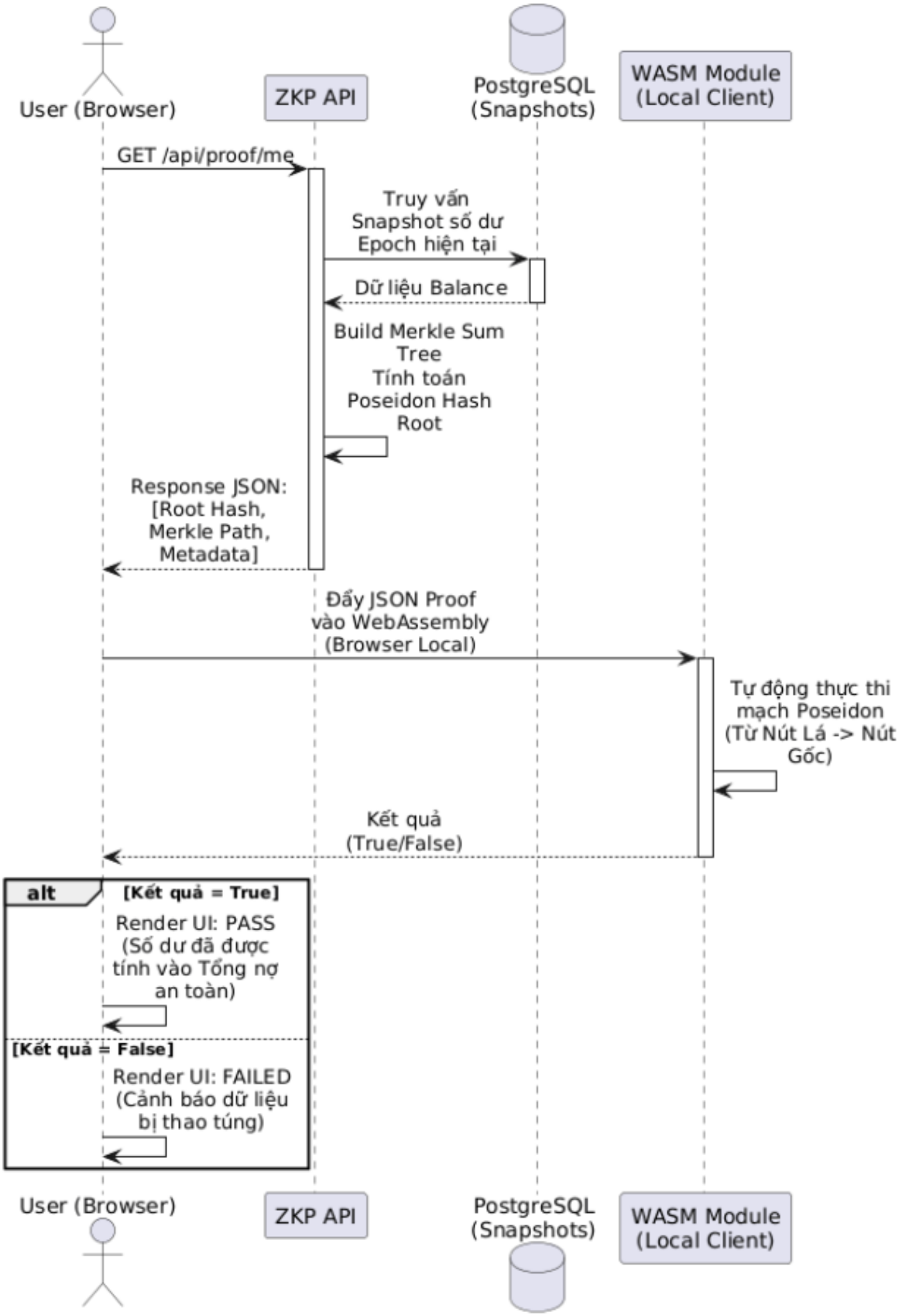
3.3.2 Quy trình Hủy lệnh & Hoàn tiền (Cancel Order)



Hình 3.2 Sơ đồ tuần tự luồng hủy lệnh

1. Người dùng gửi yêu cầu Hủy lệnh đang chờ.
2. *Engine* tìm và xóa chính xác lệnh đó khỏi cấu trúc *BTreeMap* (độ phức tạp *O(1))*.
3. Trả kết quả về cho *Ledger* để mở khóa tiền (Chuyển từ *Locked Balance trả về Free Balance*).
4. *Worker* ghi nhận trạng thái *Cancelled* xuống Cơ sở dữ liệu.

3.3.2 Quy trình Xác thực ZKP tại Client-side

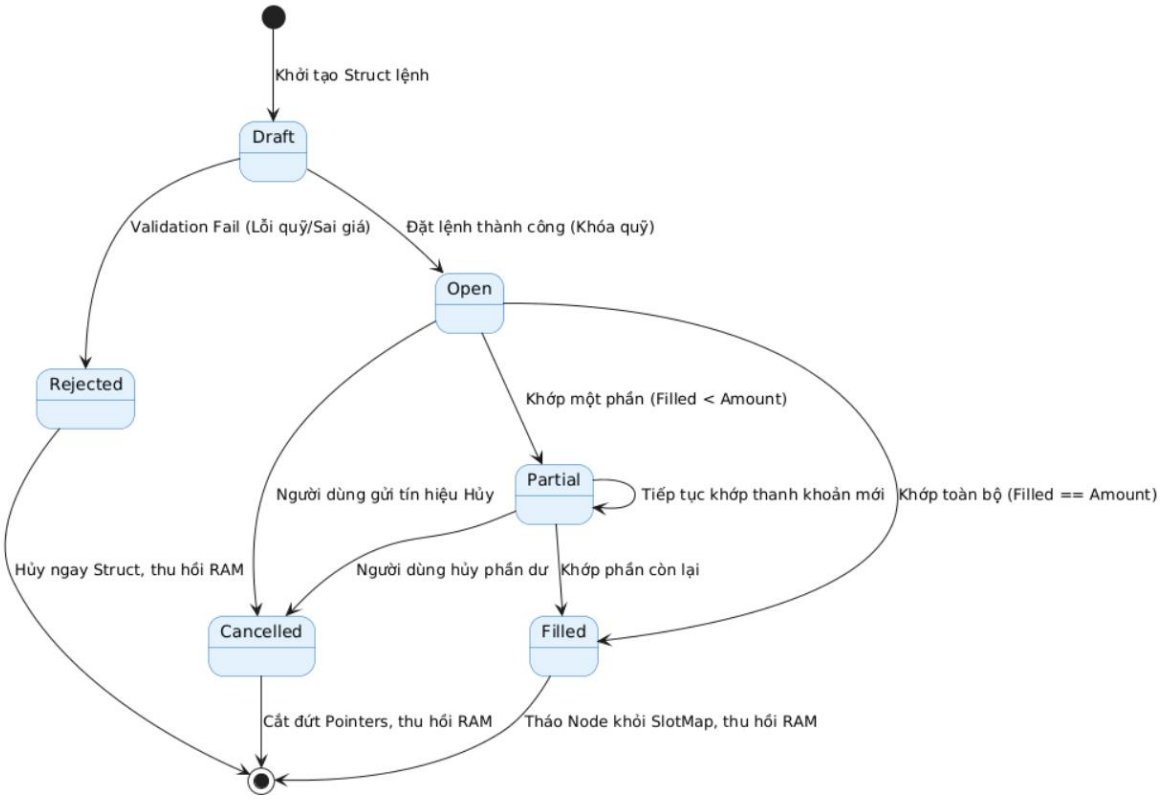


Hình 3.3 Sơ đồ tuần tự luồng ZKP

1. Người dùng vào trang "*ZK Verify*", bấm *Fetch Proof*.
2. *Server* gom dữ liệu, trả về file *JSON* chứa *Root Hash* của sàn và *Merkle Path* của người dùng đó.
3. Trình duyệt tải tệp *WASM* (*WebAssembly*) chứa mạch logic xác minh mật mã.
4. Trình duyệt tự chạy hàm băm *Poseidon* theo đường dẫn *Merkle Path*. Nếu kết quả trùng khớp với *Root Hash*, màn hình hiển thị trạng thái hợp lệ (*PASS).* Luồng này đảm bảo *Server* không thể đánh lừa người dùng.

3.3.3 Sơ đồ Trạng thái: Vòng đời Lệnh giao dịch (Order Lifecycle)

Để đảm bảo hệ thống không bị rò rỉ bộ nhớ (*Memory Leak*), số phận của một lệnh giao dịch từ khi sinh ra đến khi kết thúc được quản lý nghiêm ngặt qua một cỗ máy trạng thái (*State Machine*). Các lệnh rơi vào trạng thái cuối (*Terminal States*) như FILLED, CANCELED, hoặc REJECTED sẽ lập tức bị dọn dẹp khỏi RAM.



Hình 3.4 Sơ đồ Trạng thái của vòng đời một lệnh giao dịch

3.4 Thiết kế hệ thống

3.4.1 Thiết kế cơ sở dữ liệu

Bảng 3.1 Bảng users

STT	THUỘC TÍNH	KIỂU DỮ LIỆU	KHOÁ	MÔ TẢ
1	id	BIGINT	PK	ID người dùng (khớp với x-user-id header).
2	username	TEXT		Tên đăng nhập duy nhất.
3	display_name	TEXT		Tên hiển thị.
4	password_hash	TEXT		Mã băm của mật khẩu (Argon2id).
5	created_at	TIMESTAMPTZ		Thời gian tạo tài khoản.

Bảng 3.2 Bảng assests

STT	THUỘC TÍNH	KIỂU DỮ LIỆU	KHOÁ	MÔ TẢ
1	symbol	TEXT	PK	Mã tài sản (VD: 'BTC', 'USDT').
2	name	TEXT		Tên đầy đủ của tài sản.
3	decimals	SMALLINT		Số chữ số thập phân hỗ trợ (mặc định 8).
4	created_at	TIMESTAMPTZ		Thời gian niêm yết tài sản lên hệ thống.

Bảng 3.3 Bảng balances

STT	THUỘC TÍNH	KIỂU DỮ LIỆU	KHOÁ	MÔ TẢ
1	user_id	BIGINT	PK, FK	Tham chiếu users(id).
2	assest_symbol	TEXT	PK, FK	Tham chiếu assets(symbol).
3	available	NUMERIC(30,8)		Số dư khả dụng (tiền rảnh rỗi).
4	locked	NUMERIC(30,8)		Số dư đang bị khóa trong các lệnh chờ.

5	updated_at	TIMESTAMPTZ		Thời điểm cập nhật số dư lần cuối.

Bảng 3.4 Bảng orders_log

STT	THUỘC TÍNH	KIỂU DỮ LIỆU	KHOÁ	MÔ TẢ
1	id	UUID	PK	id log của lệnh
2	order_id	BIGINT		id lệnh
3	user_id	BIGINT	FK	Người đặt lệnh, tham chiếu users(id).
4	side	order_side		Chiều giao dịch (Buy hoặc Sell).
5	price	NUMERIC(30,8)		Mức giá đặt lệnh.
6	amount	NUMERIC(30,8)		Khối lượng đặt lệnh ban đầu.
7	filled	NUMERIC(30,8)		Khối lượng đã khớp thành công.
8	status	order_status		Trạng thái lệnh (open, filled, canceled...).
9	base_assest	TEXT		Tài sản cơ sở (VD: 'BTC').
10	quote_assest	TEXT		Tài sản định giá (VD: 'USDT').
11	created_at	TIMESTAMPTZ		Thời gian tạo lệnh.
12	updated_at	TIMESTAMPTZ		Thời gian cập nhật lệnh lần cuối.

Bảng 3.5 Bảng trades_log

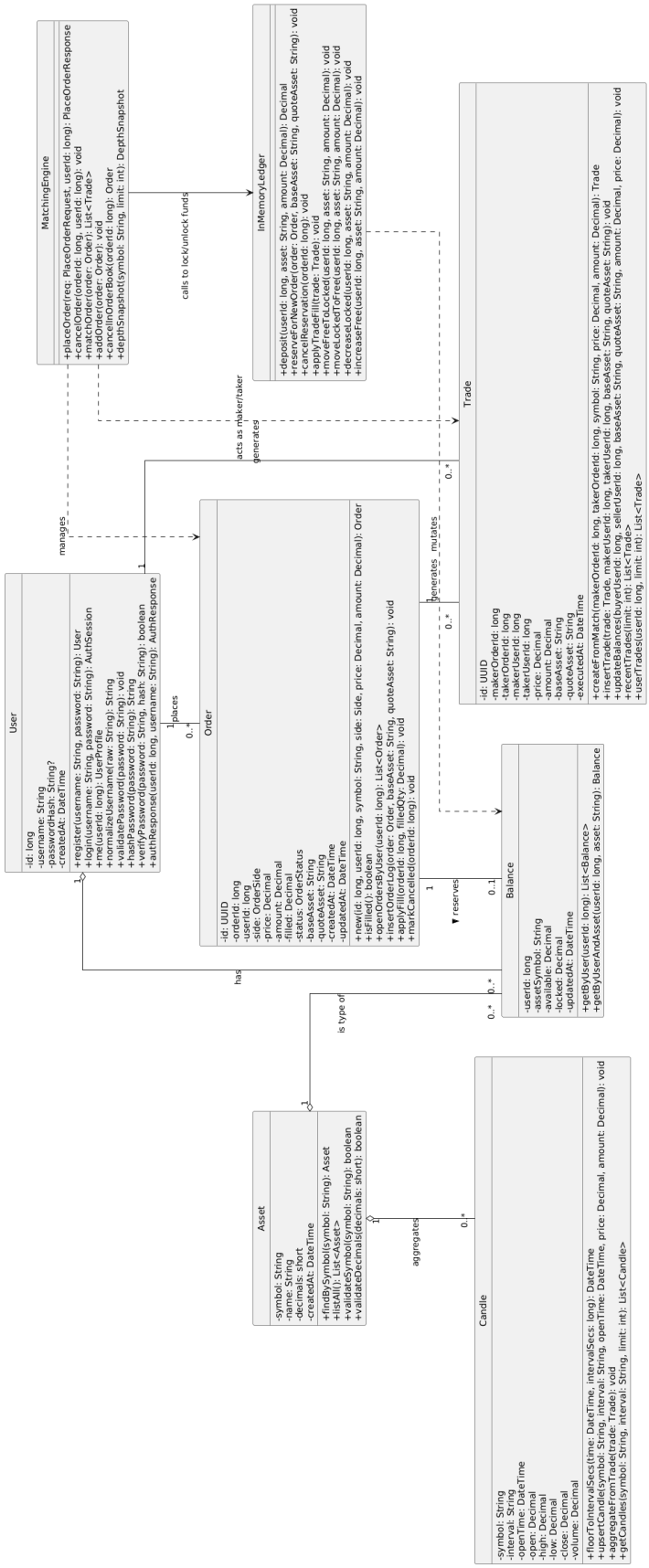
STT	THUỘC TÍNH	KIỂU DỮ LIỆU	KHOÁ	MÔ TẢ
1	id	UUID	PK	ID của giao dịch khớp lệnh.
2	maker_order_id	BIGINT	FK	Lệnh chờ bị khớp, tham chiếu orders_log(order_id).
3	taker_order_id	BIGINT	FK	Lệnh chủ động đâm vào, tham chiếu orders_log(order_id).
4	maker_user_id	BIGINT	FK	User cung cấp thanh khoản, tham chiếu users(id).
5	taker_user_id	BIGINT	FK	User lấy thanh khoản, tham chiếu users(id).
6	price	NUMERIC(30,8)		Mức giá khớp thực tế.
7	amount	NUMERIC(30,8)		Khối lượng khớp thực tế.
8	quote_asset	TEXT		Tài sản cơ sở.
9	base_asset	TEXT		Tài sản định giá.
10	executed_at	TIMESTAMPTZ		Thời điểm hai lệnh khớp nhau.

Bảng 3.6 Bảng candles

STT	THUỘC TÍNH	KIỂU DỮ LIỆU	KHOÁ	MÔ TẢ
1	symbol	TEXT	PK	Cặp giao dịch (VD: 'BTC_USDT').
2	interval	TEXT	PK	Khung thời gian ('1m', '5m', '1h', '1d').
3	open_time	TIMESTAMPTZ	PK	Mốc thời gian bắt đầu cây nến (chuẩn UTC).
4	open	NUMERIC(30,8)		Giá mở cửa.
5	high	NUMERIC(30,8)		Giá cao nhất trong khung thời gian.

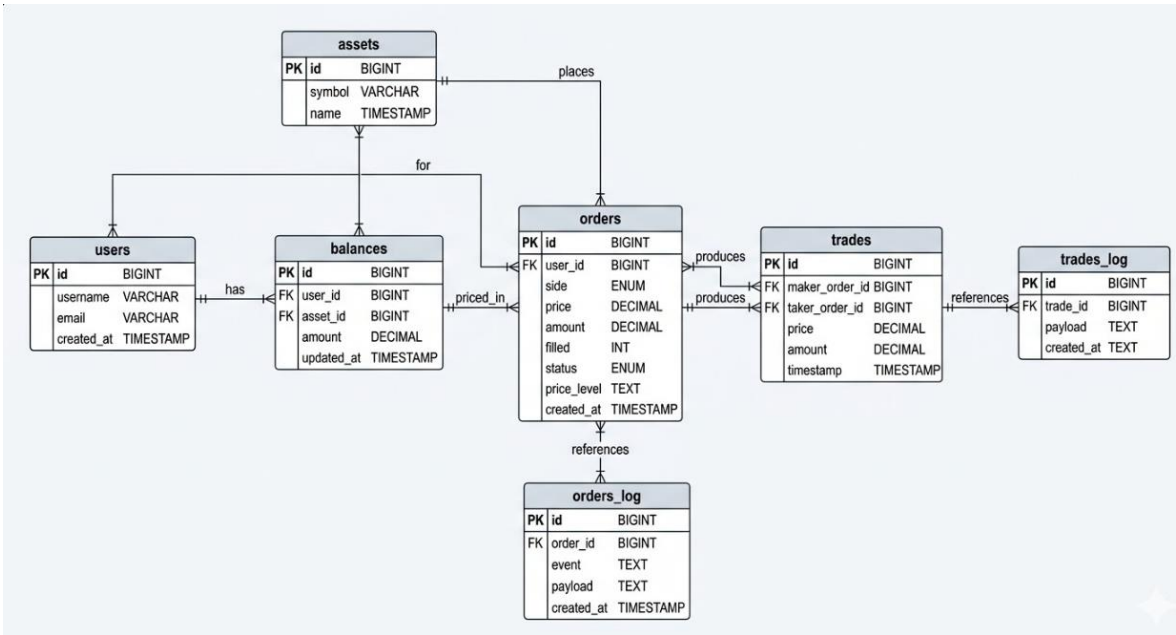
6	low	NUMERIC(30,8)		Giá thấp nhất trong khung thời gian.
7	close	NUMERIC(30,8)		Giá đóng cửa.
8	volume	NUMERIC(30,8)		Tổng khối lượng giao dịch trong khung thời gian.

3.4.2 Thiết kế sơ đồ lớp



Hình 3.5 Sơ đồ lớp

3.4.3 Mô hình ERD



Hình 3.6 Sơ đồ ERD tổng quát.

3.4.4 Thiết kế chức năng

Phân hệ Quản lý Tài khoản và Ví (*Account & Wallet Module*): Phân hệ này chịu trách nhiệm định danh người dùng và kiểm soát tính toàn vẹn của sổ cái lưu trữ tài sản (*Ledger*).

- Định danh nhanh (*Lightweight Auth*): Sử dụng cơ chế truyền *x-user-id* qua HTTP Header để xác thực luồng dữ liệu riêng tư, tối ưu hóa tốc độ xử lý thay vì dùng các quy trình giải mã JWT nặng nề.
- Quản lý trạng thái số dư: Phân tách rõ ràng số dư khả dụng (*Available/Free*) và số dư đang bị khóa (*Locked/Reserved*) khi người dùng tham gia đặt lệnh.
- Nạp tiền giả lập (*Mock Deposit*): Cung cấp cơ chế cấp vốn (*Fund*) ban đầu để người dùng có tài sản tham gia giao dịch trên môi trường thử nghiệm.

Bảng 3.7 Danh sách API Phân hệ Quản lý Tài khoản và Ví

Nghiệp vụ chính	Phương thức	Endpoint (Giao tiếp)	Mô tả
Lấy số dư	GET	/api/balances	Truy xuất ví Free và Locked của User.

Nạp tài sản	POST	/api/deposit	Cấp phát tài sản số giả lập vào ví.

Phân hệ Giao dịch lõi (*Core Trading Module*): Đây là bộ não của hệ thống, tương tác trực tiếp với lõi Matching Engine nằm trên RAM.

- Đặt lệnh Giới hạn (*Limit Order*): Tiếp nhận yêu cầu Mua/Bán. Hệ thống sẽ tự động tính toán, khóa số dư (*Reserve Funds*) tương ứng và đẩy lệnh vào Sổ lệnh (*Order Book)* bằng cấu trúc *B-Tree*.
- Hủy lệnh (*Cancel Order*): Rút lệnh đang chờ khỏi Sổ lệnh với độ phức tạp *O(1)* và lập tức giải phóng số dư (*Unlock Funds*) trả về cho người dùng.

Bảng 3.8 Danh sách API Phân hệ Giao dịch lõi

Nghiệp vụ chính	**Phương thức**	**Endpoint (Giao tiếp)**	**Mô tả**
Đặt lệnh mới	POST	/api/orders	Đẩy lệnh Mua/Bán vào Engine.
Hủy lệnh	DELETE	/api/orders/{id}	Xóa lệnh chưa khớp, hoàn tiền.
Lịch sử lệnh	GET	/api/orders	Xem danh sách lệnh của cá nhân.

Phân hệ Dữ liệu thị trường (*Market Data Module*): Phân hệ này đảm nhận việc phân phối dữ liệu công khai (*Public Data*) của sàn giao dịch đến toàn bộ người dùng với độ trễ siêu thấp. Để tránh làm nghẽn máy chủ bởi các yêu cầu HTTP liên tục (*Polling*), hệ thống sử dụng kênh truyền WebSocket dạng Pub/Sub.

- Cập nhật Sổ lệnh (*Orderbook Depth*): Đẩy (*Push*) trạng thái các mức giá *Bids/Asks* mới nhất.
- Cập nhật Khớp lệnh (*Recent Trades*): Phát sóng (*Broadcast*) thông tin mỗi khi có 2 lệnh va vào nhau thành công.
- Trích xuất Nến (*OHLCV*): Trả về dữ liệu đóng/mở cửa theo các khung thời gian để vẽ biểu đồ kỹ thuật.

Bảng 3.9 Danh sách API và WebSocket Phân hệ Dữ liệu thị trường

Nghiệp vụ chính	Phương thức	Endpoint (Giao tiếp)	Mô tả
Sổ lệnh Realtime	WebSocket	ws/orderbook	Cập nhật Bids/Asks liên tục.
Lịch sử Trade	WebSocket	ws/trades	Bắn tín hiệu khi có giao dịch mới.
Dữ liệu Biểu đồ	REST API	GET /api/candles	Cung cấp dữ liệu vẽ nến Nhật.

Phân hệ Kiểm toán Minh bạch (*ZKP Audit Module*): Phân hệ đặc thù chứng minh năng lực tài chính của sàn giao dịch mà không xâm phạm quyền riêng tư của khách hàng.

- Công bố năng lực (*Proof of Solvency*): Trích xuất Tổng tài sản (*Assets*) và Tổng nợ (*Liabilities*) thông qua *Root Hash* của *cây Merkle Sum*.
- Cung cấp Bằng chứng cá nhân (*Merkle Path*): Trả về đường dẫn băm (*Hash Path*) cụ thể cho từng người dùng, hỗ trợ họ tự xác minh số dư của mình đã được cộng gộp vào Tổng nợ một cách trung thực thông qua mã *WebAssembly* (*Client-side Verification*).

Bảng 3.10 Danh sách API Phân hệ Kiểm toán Minh bạch ZKP

Nghiệp vụ chính	Phương thức	Endpoint (Giao tiếp)	Mô tả
Proof of Solvency	GET	/api/proof/solvency	Lấy Root Hash và dữ liệu tổng sàn.
User Proof	GET	/api/proof/me	Lấy Merkle Path cá nhân để ZK Verify.

3.4.5 Thiết kế kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình client-server, trong đó tương tác giữa các phần được đảm bảo độ trễ thấp và bảo mật dữ liệu.

- Tầng Client (Giao diện người dùng): Chịu trách nhiệm hiển thị giao diện, quản lý trạng thái kết nối *WebSocket* và thực thi mã *Verifier WebAssembly* để kiểm chứng *Zero-Knowledge Proof* ngay trên máy thay vì máy chủ.
- Tầng Server (*API Gateway & Matching Engine*): Chứa các controller tiếp nhận HTTP Request và WebSocket. Nó cũng chứa Matching Engine được lock trực tiếp trên bộ nhớ để đảm bảo chỉ có 1 thread được duyệt ghi làm thay đổi cấu trúc Sổ lệnh tại 1 thời điểm.
- Tầng Persistence (*Database Worker*): Chạy như một hàng đợi xử lý nền móng, nhận các transaction sự kiện (*Event Stream*) từ tầng Engine trên RAM và ghi đồng bộ từ từ xuống ổ đĩa vật lý nhằm tránh hiện tượng "*nghẽn cổ chai*" cản trở Engine.

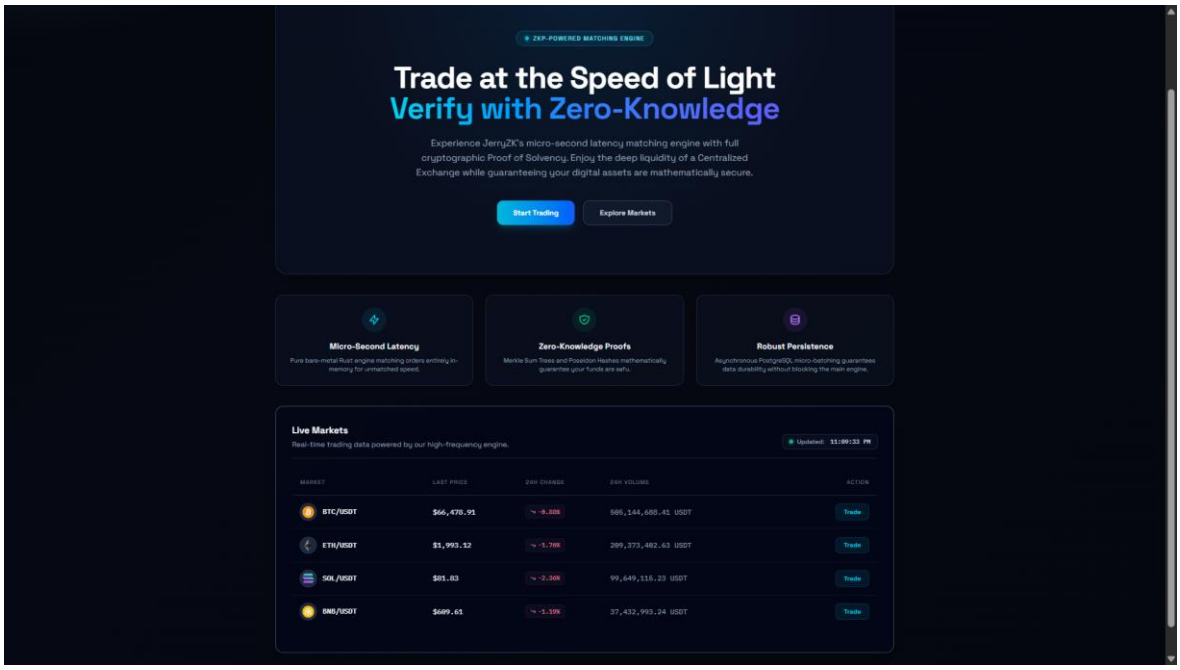
3.4.6 Thiết kế phân quyền và bảo mật

Do hệ thống hoạt động hoàn toàn trên bộ nhớ chia sẻ (*Shared Memory*), bài toán đồng bộ hóa dữ liệu đa luồng (*Concurrency*) được đặt lên hàng đầu nhằm tránh lỗi Race Condition (*Xung đột bộ nhớ*).

- Kiểm soát đồng bộ (*RwLock/Mutex*): Mã nguồn tận dụng triệt để cơ chế RwLock (*Read-Write Lock*) của ngôn ngữ Rust. Khi có yêu cầu lấy dữ liệu để vẽ biểu đồ, hệ thống cấp khóa đọc đồng thời (*Multiple Readers*) giúp tốc độ tra cứu cực nhanh. Ngược lại, khi có tín hiệu khớp lệnh, hệ thống áp đặt khóa ghi độc quyền (*Exclusive Write Lock*), đảm bảo tại một thời điểm chỉ có duy nhất một luồng (*Thread*) được phép can thiệp vào cấu trúc *SlotMap* và *BTreeMap*, đảm bảo tính nguyên tử (*Atomicity*) của giao dịch.
- Xác thực định danh: Để phục vụ môi trường thử nghiệm hiệu năng cao, hệ thống sử dụng cơ chế định danh trực tiếp qua Header *x-user-id* thay cho quy trình giải mã JWT phức tạp, nhằm tập trung toàn bộ tài nguyên vi xử lý cho Core Engine.

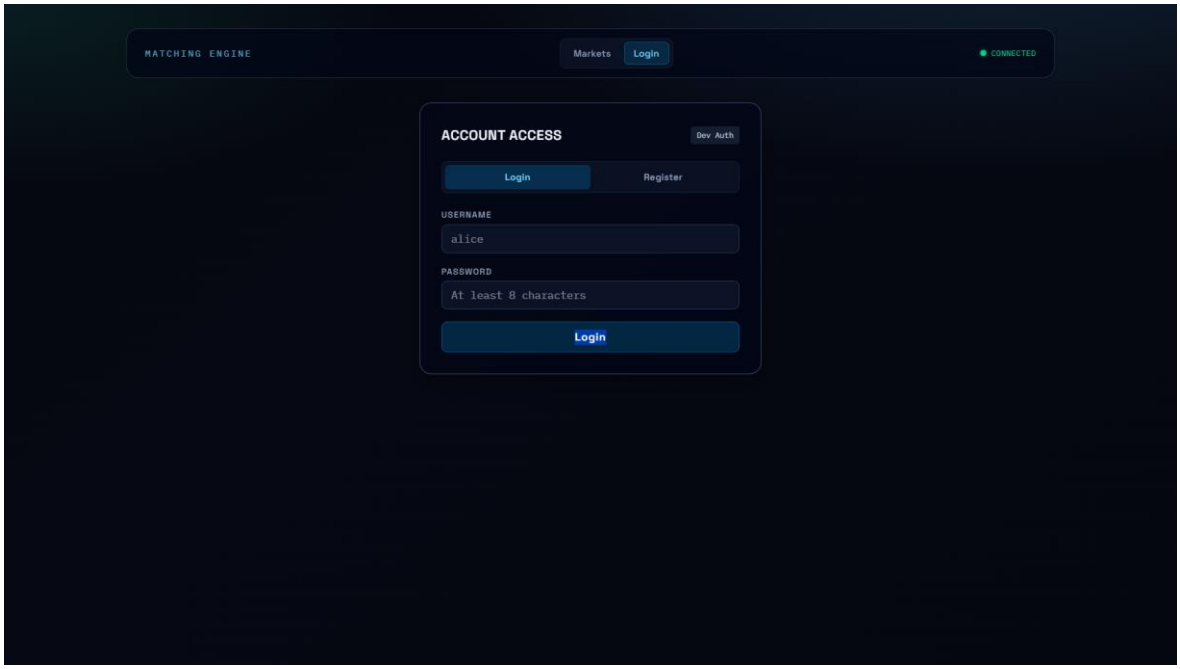
CHƯƠNG 4
GIAO DIỆN HỆ THỐNG

4.1 Giao diện Landing Page



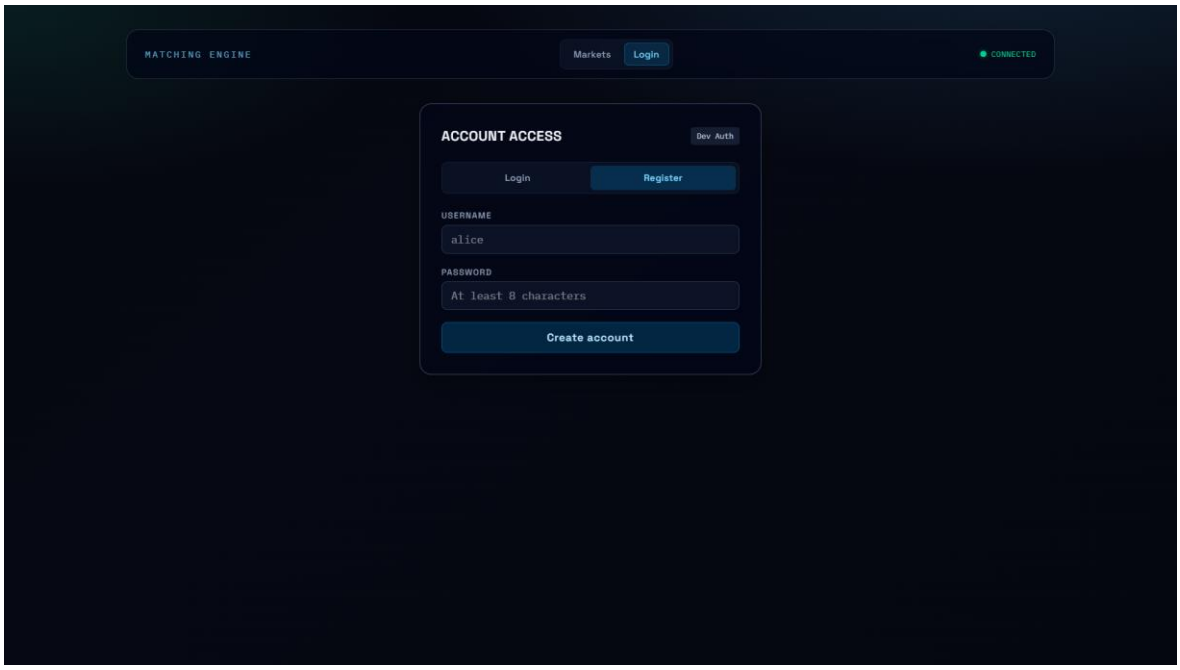
Hình 4.1 Giao diện Landing Page

4.2 Giao diện Đăng Nhập



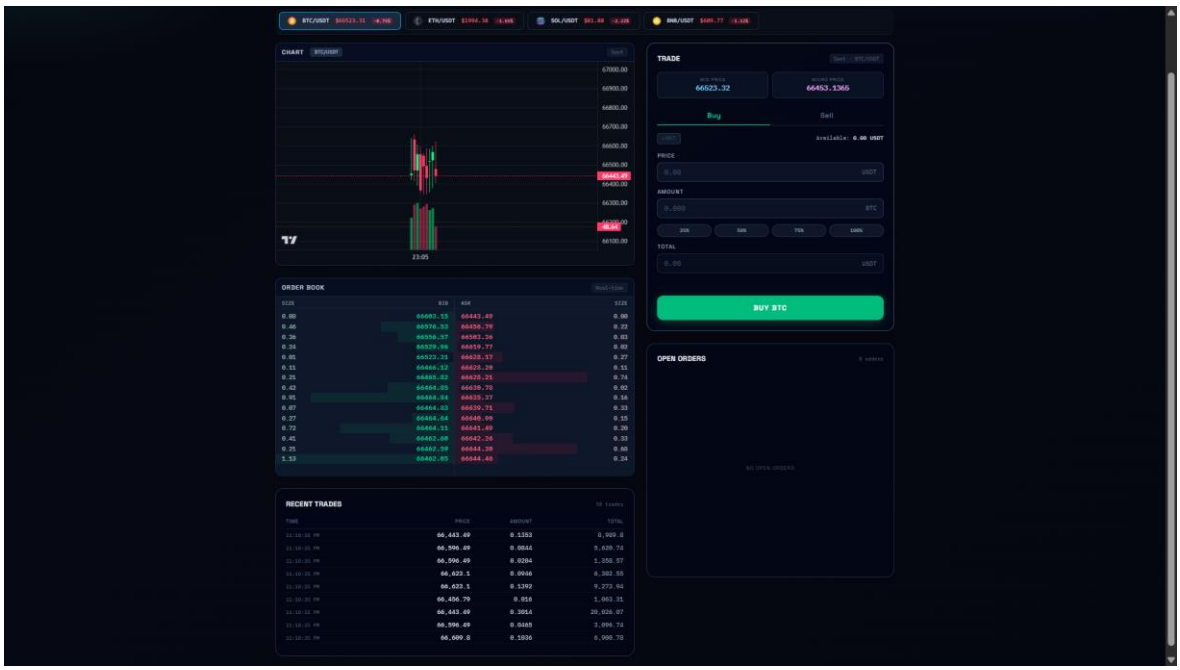
Hình 4.2 Giao diện Đăng Nhập

4.3 Giao diện Đăng ký



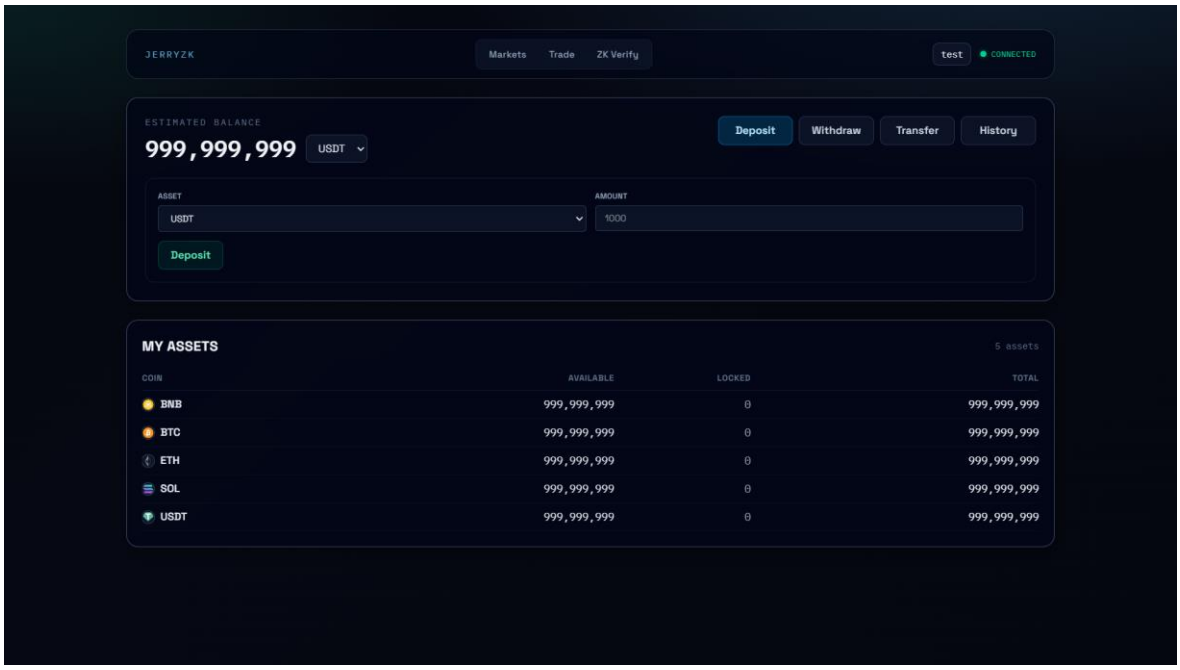
Hình 4.3 Giao diện Đăng Ký

4.4 Giao diện Trade



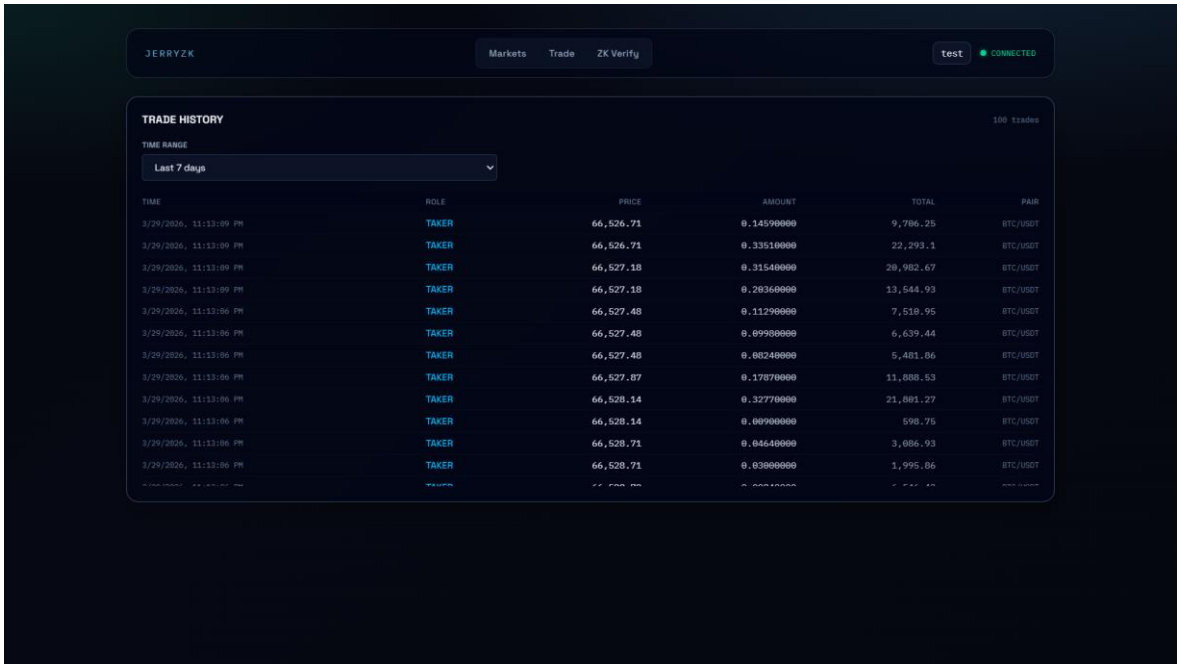
Hình 4.4 Giao diện Trade

4.5 Giao diện Asset



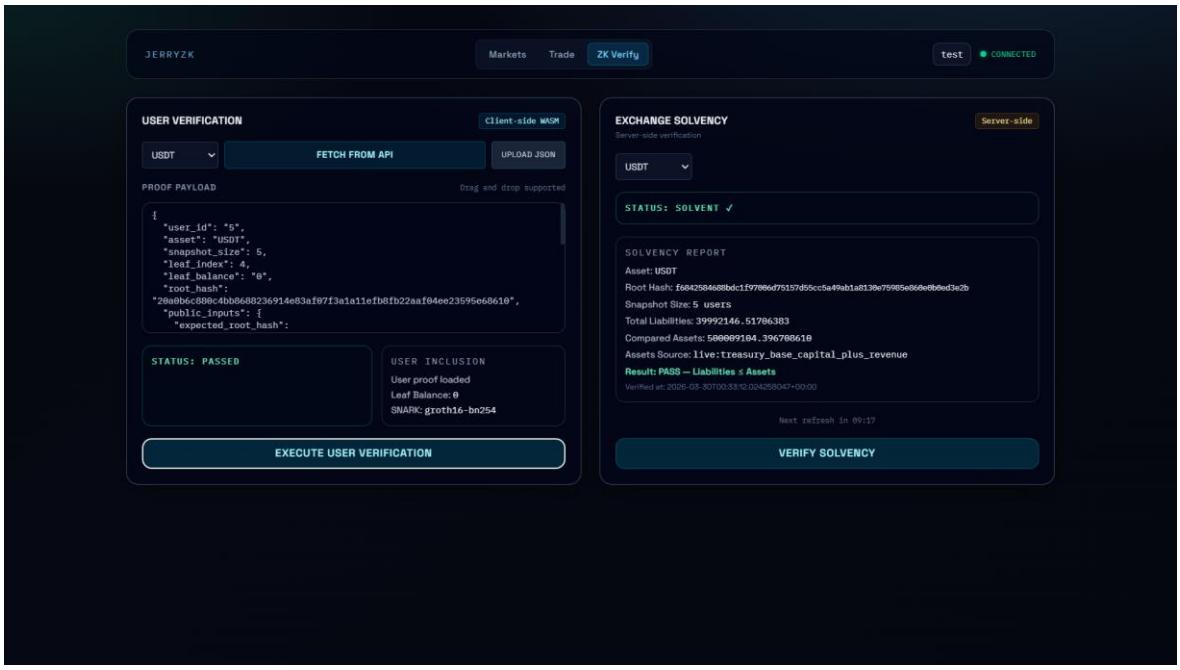
Hình 4.5 Giao diện Asset

4.6 Giao diện Trade History



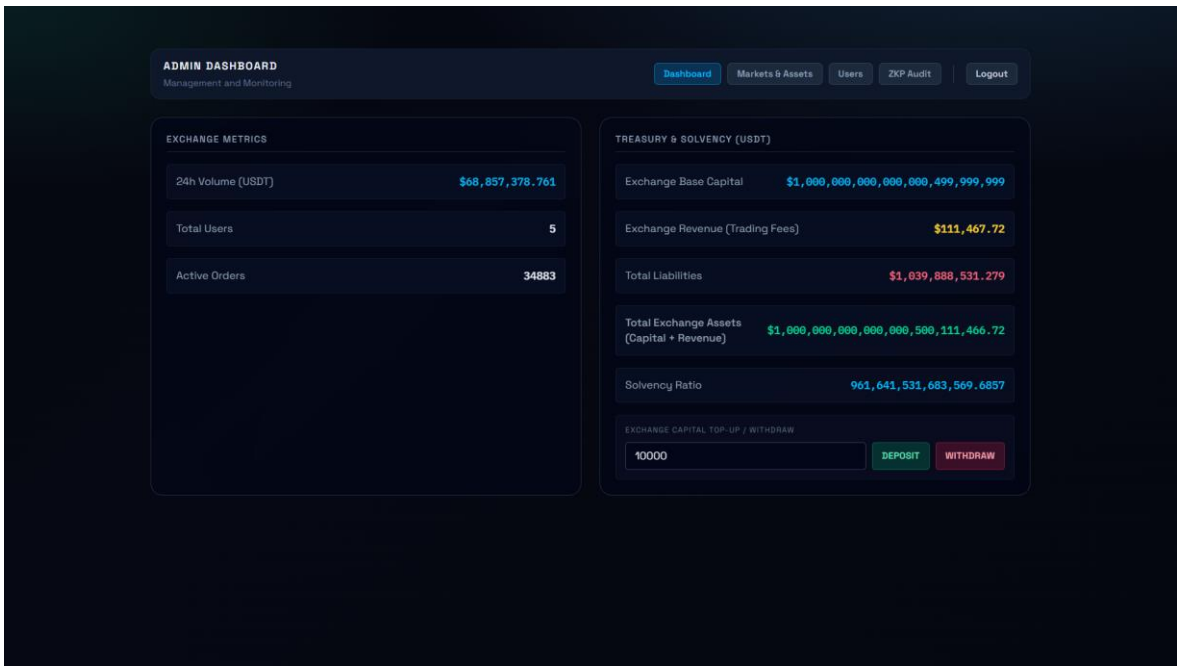
Hình 4.6 Giao diện Trade History

4.7 Giao diện ZK-Verify



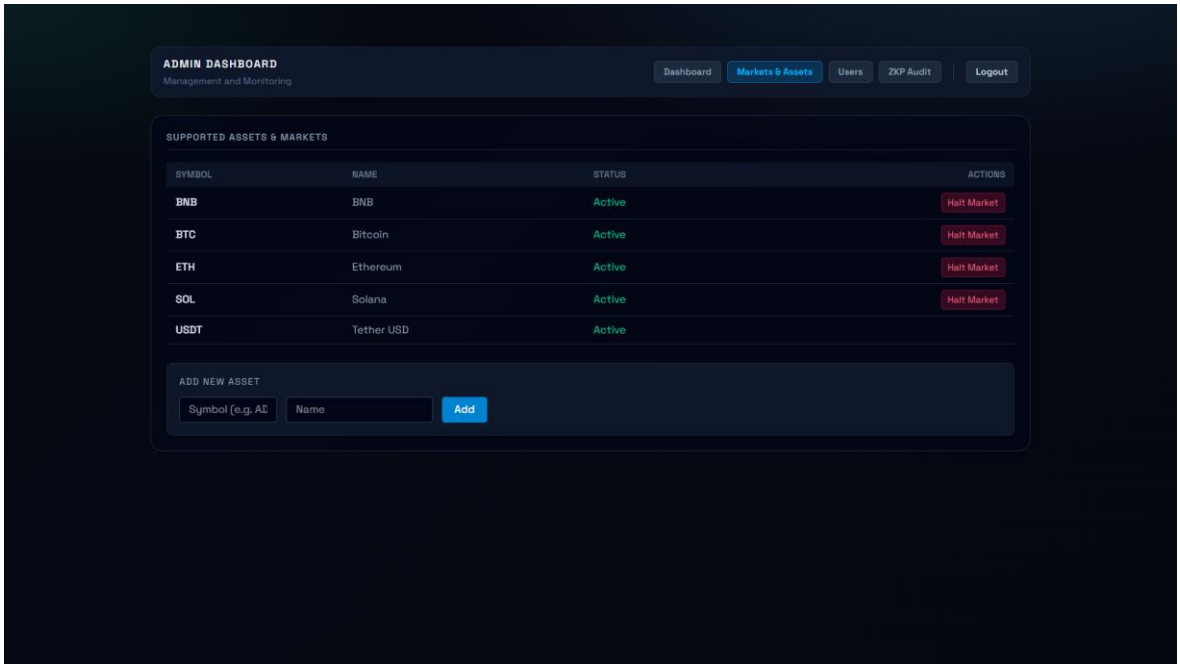
Hình 4.7 Giao diện ZK-Verify

4.8 Giao diện Dashboard Admin



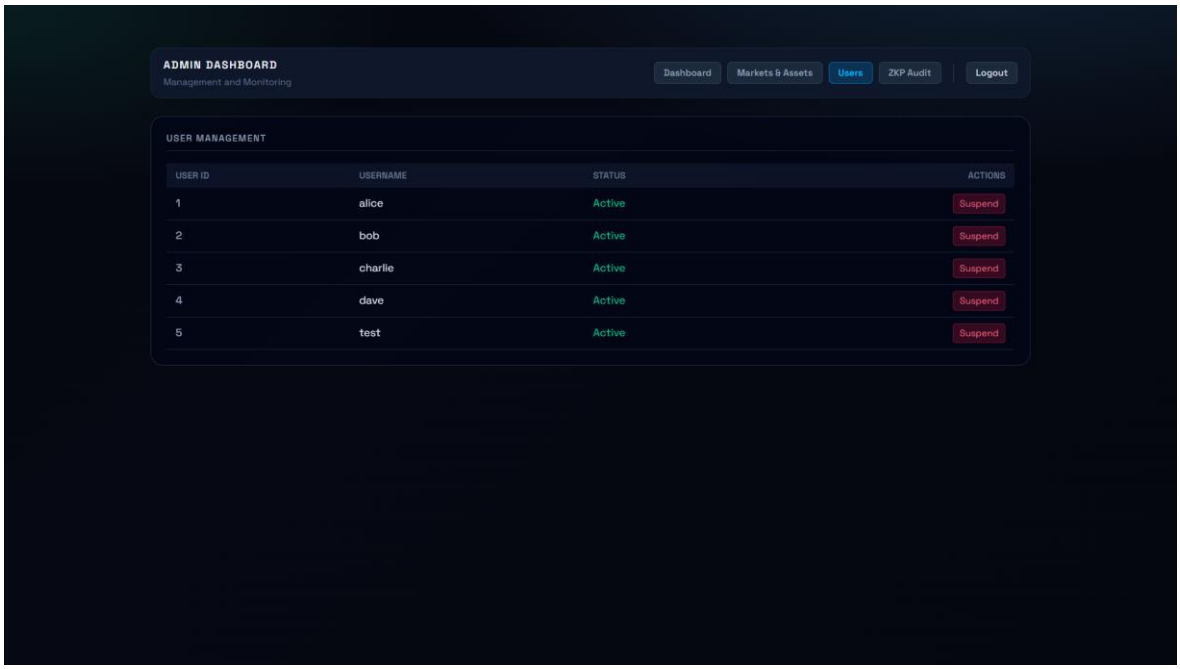
Hình 4.8 Giao diện Dashboard Admin

4.9 Giao diện Market & Asset



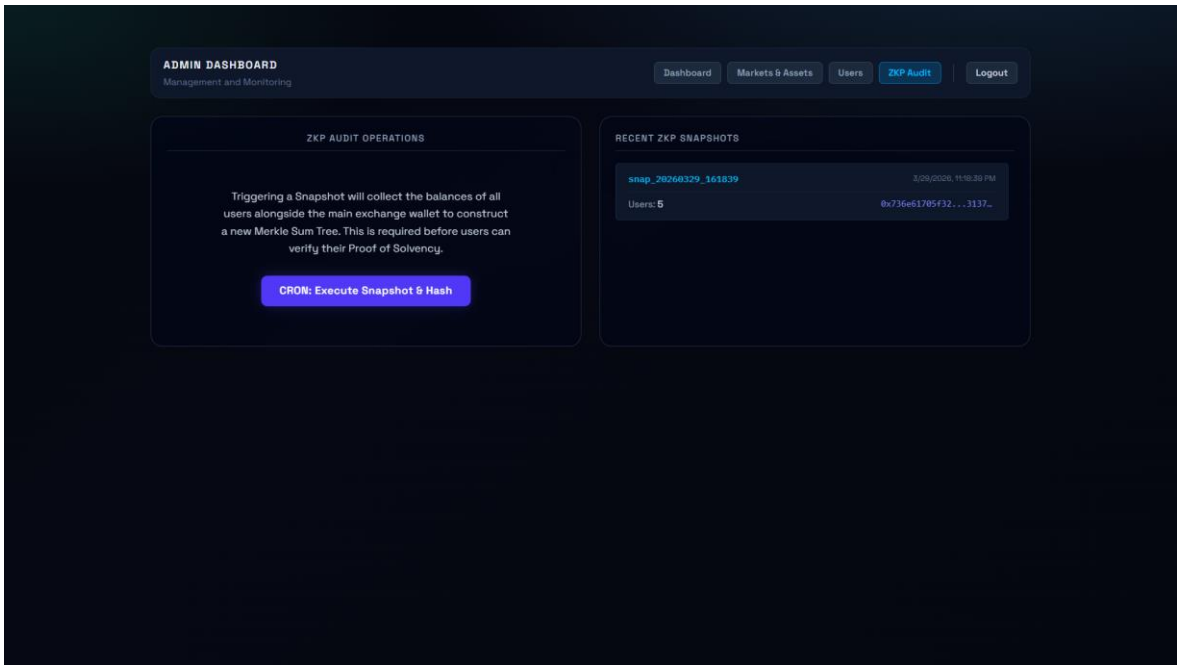
Hình 4.9 Giao diện Market & Asset

4.10 Giao diện Quản Lý User



Hình 4.10 Giao diện Quản Lý User

4.11 Giao diện ZKP Audit



Hình 4.11 Giao diện ZKP Audit

CHƯƠNG 5
KẾT LUẬN

5.1 Kết luận

Trong bối cảnh thị trường tài sản số đang giằng xé giữa hai thái cực: Sàn phi tập trung (*DEX*) minh bạch nhưng chậm chạp, và Sàn tập trung (*CEX)* tốc độ cao nhưng vận hành như một "*hộp đen*" rủi ro, đồ án đã định hình và hiện thực hóa thành công một kiến trúc lai (*Hybrid Architecture*) giải quyết triệt để nút thắt này.

Cụ thể, đồ án đã hoàn thành xuất sắc các mục tiêu cốt lõi sau:

- Thiết kế thành công Lõi khớp lệnh siêu tốc (*Ultra-low Latency Engine*): Bằng việc loại bỏ hoàn toàn cơ sở dữ liệu vật lý khỏi luồng thực thi nóng (*Hot-path*) và đưa toàn bộ Sổ cái (*Ledger*), Sổ lệnh (*OrderBook*) lên bộ nhớ RAM (*In-Memory*), hệ thống đã đạt được tốc độ khớp lệnh cực cao. Việc ứng dụng ngôn ngữ cấp hệ thống Rust kết hợp cùng các cấu trúc dữ liệu tinh vi như *SlotMap, B-Tree* và Danh sách liên kết đôi nhúng (*Intrusive DLL*) đã giúp thuật toán cốt lõi - đặc biệt là tác vụ hủy lệnh (*Cancel Order*) - đạt độ phức tạp thời gian hằng số *O(1)* tuyệt đối.
- Minh bạch hóa tài chính với *Zero-Knowledge Proof*: Đồ án đã phá vỡ rào cản mất niềm tin của mô hình *CEX* bằng cách tích hợp thành công giao thức Bằng chứng khả năng thanh toán (*Proof of Solvency*). Cấu trúc *Merkle Sum Tree* và hàm băm *Poseidon* (*ZK-Friendly*) cho phép sàn giao dịch chứng minh toán học về tính toàn vẹn của tổng tài sản dự trữ mà không làm rò rỉ bất kỳ thông tin số dư cá nhân nào.
- Phân quyền kiểm toán về phía Client (*Trustless Verification*): Việc biên dịch thành công bộ công cụ sinh/xác minh mật mã học sang định dạng *WebAssembly* (*WASM*) đã cho phép người dùng tự thực thi mã kiểm toán trực tiếp trên trình duyệt web, loại bỏ hoàn toàn sự phụ thuộc tín nhiệm vào máy chủ trung tâm.

5.2 Hạn chế

Bên cạnh những kết quả đột phá, dưới góc độ thiết kế hệ thống, kiến trúc hiện tại vẫn tồn tại một số đánh đổi bắt buộc để ưu tiên cho hiệu năng:

- *Rủi ro bốc hơi dữ liệu vùng nhớ (Volatility Risk):* Do hệ thống áp dụng cơ chế ghi chép bất đồng bộ (*Async Flushing*) từ RAM xuống PostgreSQL thông qua kênh *MPSC*, tồn tại một cửa sổ thời gian cực nhỏ (tính bằng mili-giây) dữ liệu chỉ nằm trên RAM. Nếu máy chủ vật lý gặp sự cố mất điện hoặc sập nguồn đột ngột (*Hard Crash*), các giao dịch vừa

khớp trong cửa sổ thời gian này chưa kịp ghi xuống ổ cứng sẽ bị mất vĩnh viễn, dẫn đến sai lệch trạng thái sổ cái lúc khởi động lại.

- *Giới hạn mở rộng ngang (Scale-out Bottleneck):* Mô hình Monolithic In-Memory ép buộc luồng khớp lệnh của một cặp giao dịch (Ví dụ: *BTC/USDT*) phải chạy trên một tiến trình duy nhất để đảm bảo tính tuần tự. Điều này khiến Core Engine khó mở rộng theo chiều ngang (*thêm nhiều máy chủ*) mà chỉ có thể mở rộng theo chiều dọc (*Scale-up: Nâng cấp RAM và CPU đơn nhân của máy chủ hiện tại*).

5.3 Hướng phát triển

Để nâng cấp nền tảng từ một đồ án nghiên cứu thành một sản phẩm thương mại cấp độ doanh nghiệp (*Enterprise-grade*), hệ thống có thể được mở rộng theo các hướng sau:

- Áp dụng kiến trúc *Event Sourcing* và *Write-Ahead Log (WAL):* Để giải quyết triệt để rủi ro mất dữ liệu trên RAM, hệ thống sẽ tích hợp một bộ đệm nhật ký tuần tự (như *Apache Kafka hoặc Redpanda*). Mọi lệnh đầu vào sẽ được ghi thẳng vào *WAL* (với tốc độ I/O cực nhanh) trước khi đẩy vào Engine. Nếu hệ thống sập, khi khởi động lại, Engine chỉ cần phát lại (*Replay*) toàn bộ sự kiện từ *WAL* để khôi phục trạng thái bộ nhớ chính xác 100% (*Deterministic State Rebuilding*).
- Khả dụng cao (*High Availability*) với thuật toán Đồng thuận: Triển khai thuật toán *Raft* để chạy song song nhiều Engine dưới dạng *Active-Passive Replicas*. Nếu máy chủ chính (*Leader Engine*) sập, một máy chủ phụ (*Follower*) có cùng trạng thái RAM sẽ lập tức được bầu lên thay thế, đảm bảo thời gian gián đoạn dịch vụ (*Downtime*) tiệm cận mức zero.
- Nâng cấp lên *Proof of Execution* (Bằng chứng thực thi *zk-SNARK*): Thay vì chỉ chứng minh số dư (*Proof of Solvency*), hướng đi tương lai sẽ áp dụng mạch *zk-SNARK* phức tạp hơn để chứng minh rằng mọi giao dịch khớp lệnh bên trong RAM đều tuân thủ chính xác bộ luật *Price-Time Priority* mà không có sự thiên vị hay chèn ép lệnh (*Front-running*) từ quản trị viên hệ thống.

TÀI LIỆU THAM KHẢO

1. Đặng Quốc Bảo (Chủ biên) (2020), Giáo trình cơ sở dữ liệu. Nhà xuất bản Đại học Cần Thơ, Cần Thơ.
2. Gaby G. Dagher et al. (2015), Provisions: Privacy-preserving proofs of solvency for Bitcoin exchanges. Proceedings of the 2015 ACM SIGSAC Conference on Computer and Communications Security.
3. Vitalik Buterin (2022), Having a safe CEX: proof of solvency and beyond. Ethereum Foundation Blog. (Truy cập: 15/01/2026).
4. Grassi, L. et al. (2020), Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. USENIX Security Symposium.
5. PostgreSQL Global Development Group (2024), PostgreSQL 16.0 Documentation. https://www.postgresql.org/docs/. (Truy cập: 20/01/2026).
6. TradingView (2024), Lightweight Charts Documentation. https://tradingview.github.io/lightweight-charts/. (Truy cập: 25/01/2026).
7. Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994), Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional. (Áp dụng cho Design Patterns trong kiến trúc Backend).